

XERULITH

Manuel technique et de programmation

Profils de robot, protocoles, modèles d'IA, missions et scripts — version 0.4.0

JD-Humanoid · hexapodes et quadrupèdes EZ-B · rovers · ArduPilot (ArduRover, ArduSub, bateau) · cartes maison · SunFounder GalaxyRVR · drone et rover Arma 3

by F1GBD (c) 2026 — projet de R&D personnel

Octobre 2026

Sommaire

1. Objet du document....4
1.1 Conventions....4
1.2 Vue d'ensemble....4
1.3 Pilotage autonome par IA et simulation Arma 3....4
2. Les profils de robot....6
2.1 Structure....6
2.2 Types de robot....7
2.3 Contrôleurs....7
2.4 Modes de déplacement....7
2.5 Capteurs et sources....8
2.6 Stockage et profils fournis....10
3. Programmer un modèle de robot....11
3.1 JD-Humanoid (XERULITH)....11
3.2 Hexapode et quadrupède EZ-B (actions ARC)....11
3.3 Rover à servos à rotation continue....12
3.4 Rover à pont en H (L298N, TB6612)....12
3.5 Tourelle de caméra (pan / tilt)....12
3.6 ArduRover (rover ArduPilot)....13
3.7 ArduSub (sous-marin)....14
3.8 Bateau ArduPilot....14
3.9 Carte maison (ESP32, Arduino) en TCP ou série....14
3.10 SunFounder GalaxyRVR....15
3.11 Simulateur....16
3.12 Drone Arma 3 (Pythia)....16
3.13 Rover UGV Arma 3 (Pythia)....19
4. Protocoles de communication....21
4.1 EZ-B v4....21
4.2 MAVLink (ArduPilot)....21
4.3 Carte maison....22
4.4 SunFounder GalaxyRVR....22
4.5 Flux vidéo et GPS....22
4.6 Pont Arma 3 (Pythia)....23
5. Les modèles d'IA....28
5.1 Connexion à Ollama....28
5.2 Rôles des modèles....28
5.3 Paramètres des requêtes....28
5.4 Invite système du pilote....29
5.5 Format de réponse et directives....29
5.6 Garde-fous de l'exploration....30
5.7 Modèle de vision....30
6. Les missions d'exploration....31
6.1 Bibliothèque de missions....31
6.2 Missions fournies....31
6.3 Écrire une mission....34
6.4 Observation reçue par le pilote....35
6.5 Exemples par type de robot....36
6.6 Réglages de conduite....37
6.7 Journal et rapport....37
7. Programmation par scripts Python....38
7.1 Référence de l'API....38
7.2 Exemples....39
8. Commandes vocales....40
9. Sécurité....41

10. Fichiers et réglages.....42
11. Diagnostic.....43
Annexe A. Bibliothèque du JD.....44
Annexe B. Crédits et licences.....45

1. Objet du document

Ce manuel s'adresse à l'opérateur qui veut aller au-delà de l'utilisation courante de **XERULITH** : comprendre comment il pilote ses différentes plateformes (EZ-Robot, ArduPilot / MAVLink, SunFounder GalaxyRVR, cartes maison, drone et rover Arma 3), décrire et programmer un nouveau robot, raccorder un contrôleur ArduPilot ou une carte maison, régler les modèles d'IA et écrire des missions d'exploration autonome. Il ne demande ni les sources ni Python : tout se fait dans l'application installée, ses profils, ses missions et ses scripts. La compilation, l'architecture du code et l'extension du programme sont décrites dans le **Manuel du développeur**. Le **Manuel de l'utilisateur** reste la référence pour la prise en main de l'interface.

1.1 Conventions

- Les noms de fichiers, de champs JSON, de commandes et de directives sont écrits `ainsi`.
- Les chemins relatifs partent du dossier de l'application ; `xerulith_data` est le dossier de données placé à côté de `XERULITH.exe` (menu **Fichier → Ouvrir le dossier de données**).
- Les valeurs par défaut citées sont celles de la version 0.4.0 .
- Les servos sont désignés par leur port EZ-B (D0 à D23), les angles en degrés (1 à 180), les distances en centimètres et les tensions en volts.

1.2 Vue d'ensemble

XERULITH est une plateforme de pilotage et d'exploration autonome **multi-robots** pour Windows. Il pilote un robot décrit par un **profil**, analyse l'image de sa caméra, parle par une synthèse vocale, exécute des scripts Python et confie, sur demande, le pilotage à un **grand modèle de langage** servi par Ollama, en local ou dans le cloud. Le cas de référence est le robot humanoïde XERULITH (base EZ-Robot JD-Humanoid) ; les autres robots sont pris en charge par profils.

Fonction	Mise en œuvre
Robots EZ-Robot	Protocole EZ-B v4 (projet Commander, licence MIT, sans modification) : JD-Humanoid et sa bibliothèque de mouvements, hexapodes et quadrupèdes (actions ARC), rovers à servos continus ou à pont en H
Véhicules et autres cartes	Profils JSON et pilotes : ArduPilot / MAVLink (ArduRover, ArduSub, bateau), SunFounder GalaxyRVR, carte maison série ou TCP, simulateur intégré
Simulation de vol	Drone AR-2 Darter dans Arma 3 : mod Pythia, pont `@XerulithBridge` (Python dans le jeu) et script de mission SQF ; vidéo par capture de la fenêtre du jeu
Vision	OpenCV (couleur, QR, visages, ArUco, mouvement), SSD MobileNet v1 COCO par onnxruntime, modèle vision d'Ollama
IA	API HTTP d'Ollama : dialogue, directives de pilotage, boucle d'exploration autonome, rapport de mission
Voix	Synthèse Windows SAPI ou Piper, reconnaissance faster-whisper (facultative)
Automatisation	Scripts Python complets avec l'objet `robot`, commandes vocales paramétrables

1.3 Pilotage autonome par IA et simulation Arma 3

La boucle d'exploration confie le pilotage **complet** du robot à des modèles servis par Ollama : un modèle de vision décrit la scène, un modèle de décision choisit les actions à partir de la mission écrite en français, des mesures des capteurs et de sa mémoire, puis rédige le rapport. Chaque ordre reste vérifié par XERULITH (chapitre 5). Le même pilote IA commande toutes les plateformes ; seule la description du profil change.

Ollama	Usage conseillé
Local (PC ou serveur du	autonomie complète sans Internet, données et images sur place, latence

Ollama	Usage conseillé
réseau)	maîtrisée
Cloud (ollama.com, clé API)	grands modèles sans GPU local ; même interface et mêmes garde-fous
Automatique	réseau local, puis cloud en secours

Le drone virtuel d'**Arma 3** sert de banc d'essai et de formation : une mission se met au point en simulation (texte, limites, choix du modèle), avec des exercices générés au hasard et évalués (balise 121,5 MHz, départ de feu, reconnaissance), avant d'être rejouée sur un robot ou un véhicule réel (§ 3.12 et 4.6).

2. Les profils de robot

Un profil décrit **tout** ce que XERULITH doit savoir d'un robot. C'est un fichier JSON de `xerulith_data\profiles\`, un par robot, nommé d'après son identifiant. Il se modifie dans l'éditeur (bouton ⚙️) ou à la main ; dans ce cas, l'onglet JSON de l'éditeur et la commande **Appliquer le JSON à la fiche** permettent de vérifier le résultat.

2.1 Structure

```
{
  "version": 1,
  "id": "mon_robot",
  "nom": "Mon robot",
  "type": "rover",
  "description": "rover à quatre roues avec une caméra orientable",
  "controleur": { "type": "ezb", "params": { "adresse": "192.168.1.65" } },
  "deplacement": {
    "type": "servos_continus",
    "pas_cm": 10,
    "rotation_deg": 30,
    "params": { "…": "…" }
  },
  "tete": {
    "pan": 2,
    "tilt": 3,
    "centre_pan": 90,
    "centre_tilt": 90,
    "balayage": 45
  },
  "servos": [{ "port": 2, "nom": "Caméra — panoramique", "min": 10, "max": 170 }],
  "capteurs": [
    {
      "type": "distance",
      "nom": "Ultrason avant",
      "source": "ezb_adc",
      "params": { "port": 0, "cm_par_unite": 2 }
    }
  ],
  "fonctions": {
    "calibrer": false,
    "actions_jd": false,
    "yeux": false,
    "pinces": false,
    "voix": true
  },
  "securite": { "batterie_min": 7, "distance_arret_cm": 25 }
}
```

Champ	Signification
`id`	Identifiant unique (minuscules, chiffres, _) ; donne le nom du fichier ; ne change pas quand on renomme le profil
`nom`, `type`, `description`	Nom affiché ; type de robot ; phrase lue par l'IA pour savoir quel corps elle commande
`controleur`	Type de contrôleur et paramètres de connexion (§ 2.3)
`deplacement`	Mode de déplacement, distance d'un pas (`pas_cm`), rotation d'un pas (`rotation_deg`), paramètres (§ 2.4)
`tete`	Ports panoramique et inclinaison (l'un ou l'autre peut valoir `null`), centres, amplitude du balayage d'exploration ; `null` si pas de tête
`servos`	Servos EZ-B : port, nom, course `min` / `max` ; une position hors course est **refusée**

Champ	Signification
`capteurs`	Liste ordonnée : type, nom, source, paramètres (§ 2.5) ; la première caméra est celle affichée
`fonctions`	`calibrer` (calibrage obligatoire), `actions_jd` (gestes de l'IA), `yeux`, `pinces`, `voix` (haut-parleur de l'EZ-B)
`securite`	`batterie_min` (V, 0 = sans contrôle) ; `distance_arret_cm` (exploration : AVANCER refusé en deçà)

Un profil incomplet est complété à la lecture : les clés absentes prennent leur valeur par défaut, ce qui permet d'écrire des profils minimaux à la main.

2.2 Types de robot

Clé type	Libellé
`humanoide`	Humanoïde (bipède)
`hexapode`	Hexapode (6 pattes)
`quadrupede`	Quadrupède (4 pattes)
`rover`	Rover à roues ou chenilles
`sous_marin`	Sous-marin (ROV)
`bateau`	Bateau
`drone`	Drone (multirotor)
`autre`	Autre

Les types `sous_marin` et `drone` ajoutent les directions **monter** et **descendre** (boutons, touches Page précédente / Page suivante, directives `MONTER` et `DESCENDRE`).

2.3 Contrôleurs

Clé	Contrôleur	Paramètres et valeurs par défaut
`ezb`	EZ-B v4 (Wi-Fi, ports 23/24)	`adresse` = "192.168.1.1"
`mavlink`	ArduPilot / MAVLink (ArduRover, ArduSub, bateau)	`connexion` = "udp:0.0.0.0:14550" ; `baud` = 57600 ; `mode_pilotage` = "MANUAL"
`serie`	Carte maison — liaison série (USB/COM)	`port` = "COM5" ; `baud` = 115200
`tcp`	Carte maison — TCP / Wi-Fi	`adresse` = "192.168.4.1:8023"
`galaxyrvr`	SunFounder GalaxyRVR (Wi-Fi, firmware d'origine)	`adresse` = "192.168.4.1" ; `protocole` = "auto"
`arma3`	Arma 3 — drone ou rover virtuel (pont Pythia)	`adresse` = "127.0.0.1:47800" ; `origine_lat` = 48.5436 ; `origine_lon` = 3.0036
`sim`	Simulateur générique (sans matériel)	—

L'adresse saisie dans la barre de connexion remplace celle du profil quand elle n'est pas vide ; chaque profil garde sa propre adresse dans les réglages.

2.4 Modes de déplacement

Clé	Mode	Paramètres et valeurs par défaut
`jd_actions`	Actions du JD (bibliothèque Commander : Forward, Left…)	`avant` = "Forward" ; `arriere` = "Reverse" ; `gauche` = "Left" ; `droite` = "Right"

Clé	Mode	Paramètres et valeurs par défaut
`actions`	Actions d'une bibliothèque (Auto Position, fichier actions.json)	`bibliotheque` = "" ; `avant` = "Forward" ; `arriere` = "Reverse" ; `gauche` = "Left" ; `droite` = "Right"
`servos_continus`	Deux servos à rotation continue (EZ-B)	`port_gauche` = 0 ; `port_droite` = 1 ; `neutre` = 90 ; `amplitude` = 40 ; `inverser_gauche` = false ; `inverser_droite` = true ; `duree_pas_s` = 0.6
`hbridge`	Pont en H : 4 sorties numériques + 2 PWM (EZ-B)	`a1` = 0 ; `a2` = 1 ; `b1` = 2 ; `b2` = 3 ; `pwm_a` = 4 ; `pwm_b` = 5 ; `vitesse` = 70 ; `duree_pas_s` = 0.5
`mavlink`	Consignes RC ArduPilot (RC override)	`canal_avant` = 3 ; `canal_rotation` = 1 ; `canal_vertical` = 0 ; `pwm_neutre` = 1500 ; `amplitude` = 200 ; `avance_en_virage` = 0 ; `duree_pas_s` = 1
`texte`	Commandes texte vers la carte maison	`avant` = "F {v} {ms}" ; `arriere` = "B {v} {ms}" ; `gauche` = "L {v} {ms}" ; `droite` = "R {v} {ms}" ; `monter` = "U {v} {ms}" ; `descendre` = "D {v} {ms}" ; `stop` = "S" ; `vitesse` = 60 ; `duree_pas_ms` = 600
`galaxyrvr`	Moteurs gauche / droite du GalaxyRVR	`vitesse` = 60 ; `vitesse_rotation` = 70 ; `duree_pas_s` = 0.5 ; `duree_rotation_s` = 0.35
`arma3`	Consignes de vitesse du drone ou du rover Arma 3	`vitesse_ms` = 3 ; `vitesse_verticale_ms` = 1.5 ; `rotation_degs` = 45 ; `duree_pas_s` = 1 ; `vitesse_transit_ms` = 0 ; `rayon_max_m` = 6000 ; `hauteur_max_m` = 150
`sim`	Simulateur (déplacement virtuel)	—
`aucun`	Aucun déplacement (robot fixe)	—

Contrôleur	Déplacements compatibles
`ezb`	`jd_actions`, `actions`, `servos_continus`, `hbridge`, `aucun`
`mavlink`	`mavlink`, `aucun`
`serie`, `tcp`	`texte`, `aucun`
`galaxyrvr`	`galaxyrvr`, `aucun`
`arma3`	`arma3`, `aucun`
`sim`	`sim`, `aucun` (la case Simulateur simule aussi tout profil non JD)

2.5 Capteurs et sources

Capteur	Source	Paramètres
Caméra	`ezb` — Caméra EZ-B (port 24)	—
Caméra	`url` — Flux réseau (MJPEG, RTSP, HTTP)	`url` = "http://192.168.1.50:8080/video"
Caméra	`usb` — Caméra USB / webcam du PC	`index` = 0
Caméra	`sim` — Caméra de synthèse	—
Caméra	`galaxyrvr` — Caméra du GalaxyRVR (ESP32-CAM)	`url` = ""
Caméra	`ecran` — Capture d'écran (fenêtre d'un simulateur)	`fenetre` = "Arma 3" ; `ips` = 15 ; `largeur` = 960 ; `incrustation` =

Capteur	Source	Paramètres
		true
Distance (ultrason / IR)	`ezb_adc` — Capteur analogique sur ADC EZ-B (MaxBotix, Sharp…)	`port` = 0 ; `cm_par_unite` = 2
Distance (ultrason / IR)	`ezb_i2c_srf` — Ultrason I2C SRF02/SRF08/SRF10 sur EZ-B	`adresse` = 112
Distance (ultrason / IR)	`mavlink` — DISTANCE_SENSOR (ArduPilot)	—
Distance (ultrason / IR)	`texte` — Valeur envoyée par la carte maison	`cle` = "DIST"
Distance (ultrason / IR)	`galaxyrvr` — Ultrason du GalaxyRVR	—
Distance (ultrason / IR)	`arma3` — Télémètre du drone Arma 3	—
Distance (ultrason / IR)	`sim` — Simulateur	—
GPS	`ezb_uart` — GPS NMEA sur UART de l'EZ-B	`uart` = 0 ; `baud` = 9600
GPS	`serie_pc` — GPS NMEA sur port série du PC	`port` = "COM6" ; `baud` = 9600
GPS	`tcp_nmea` — Flux NMEA TCP (GPS réseau, téléphone…)	`adresse` = "192.168.1.60:10110"
GPS	`mavlink` — GPS ArduPilot	—
GPS	`texte` — Position envoyée par la carte maison	`cle` = "GPS"
GPS	`arma3` — Position du drone Arma 3	—
GPS	`sim` — Simulateur	`lat` = 48.5436 ; `lon` = 3.0036
Batterie	`ezb` — Tension mesurée par l'EZ-B	—
Batterie	`ezb_adc` — Pont diviseur sur ADC EZ-B	`port` = 1 ; `volts_par_unite` = 0.0645
Batterie	`mavlink` — SYS_STATUS (ArduPilot)	—
Batterie	`texte` — Valeur de la carte maison	`cle` = "BAT"
Batterie	`galaxyrvr` — Batterie du GalaxyRVR	—
Batterie	`sim` — Simulateur	—
Cap / boussole	`mavlink` — Cap ArduPilot	—
Cap / boussole	`texte` — Valeur de la carte maison	`cle` = "CAP"
Cap / boussole	`arma3` — Cap du drone Arma 3	—
Cap / boussole	`sim` — Simulateur	—
Profondeur	`mavlink` — Profondeur ArduSub	—
Profondeur	`texte` — Valeur de la carte maison	`cle` = "PROF"
Profondeur	`sim` — Simulateur	—
Entrée analogique	`ezb_adc` — ADC EZ-B	`port` = 2 ; `echelle` = 1 ; `unite` = ""
Entrée analogique	`texte` — Valeur de la carte maison	`cle` = "A0"
Entrée analogique	`arma3` — Mesure du drone Arma 3 (hauteur,	`mesure` = "hauteur" ; `unite` = "m"

Capteur	Source	Paramètres
	vitesse, carburant, dégâts)	
Entrée numérique (contact, fin de course)	`ezb_digital` — Entrée numérique EZ-B	`port` = 23
Entrée numérique (contact, fin de course)	`texte` — Valeur de la carte maison	`cle` = "D0"
Détecteur d'obstacle (infrarouge, contact)	`ezb_digital` — Entrée numérique EZ-B	`port` = 7 ; `actif_bas` = true
Détecteur d'obstacle (infrarouge, contact)	`texte` — Valeur de la carte maison (1 = obstacle)	`cle` = "IR"
Détecteur d'obstacle (infrarouge, contact)	`galaxyrvr` — Capteur infrarouge du GalaxyRVR	`cote` = "gauche"
Détecteur d'obstacle (infrarouge, contact)	`sim` — Simulateur	`cote` = "gauche"

Les mesures sont lues à chaque télémétrie (toutes les 2 s) et à chaque cycle d'exploration. Les sources EZ-B ne sont pas interrogées pendant une allure du JD, pour ne pas perturber son minutage : la dernière valeur est alors conservée.

2.6 Stockage et profils fournis

- Les profils fournis sont écrits au premier lancement ; ceux d'une nouvelle version apparaissent automatiquement à la mise à jour.
- Un profil fourni supprimé ne revient pas, sauf avec **Restaurer les profils fournis** ; le profil du JD est toujours recréé s'il manque.
- Le fichier témoin `.profils_fournis_v1` liste les profils déjà livrés.

Identifiant	Nom	Contrôleur / déplacement
`jd_humanoid`	JD Humanoid (XERULITH)	ezb / jd_actions
`hexa_rover`	Hexa Rover (EZ-B, hexapode)	ezb / actions
`quad_rover`	Quad Rover (EZ-B, quadrupède)	ezb / actions
`rover_roues`	Rover à roues (EZ-B)	ezb / servos_continus
`ardurover`	Rover ArduPilot (ArduRover)	mavlink / mavlink
`sub_rover`	Sub Rover (ArduSub)	mavlink / mavlink
`boat_rover`	Boat Rover (ArduPilot bateau)	mavlink / mavlink
`carte_maison`	Rover carte maison (ESP32, TCP)	tcp / texte
`galaxy_rvr`	SunFounder GalaxyRVR	galaxyrvr / galaxyrvr
`drone_arma3`	Drone Arma 3 (simulation de formation)	arma3 / arma3
`ugv_arma3`	Rover UGV Arma 3 (simulation de formation)	arma3 / arma3
`rover_sim`	Rover simulé (sans matériel)	sim / sim

3. Programmer un modèle de robot

Ce chapitre donne, pour chaque famille de robots, le profil à écrire et les points de réglage. La méthode est toujours la même : partir du profil fourni le plus proche, le **dupliquer**, ajuster ports, canaux et capteurs, puis valider pas à pas, roues levées.

1. Dupliquer le profil fourni le plus proche, le renommer, écrire sa description (une phrase pour l'IA).
2. Régler le contrôleur et tester la connexion seule (onglet Capteurs : les mesures arrivent).
3. Déclarer les servos et leur course, puis tester chaque curseur du panneau Servos.
4. Régler le déplacement : sens des moteurs, vitesse, durée d'un pas ; mesurer la distance d'un pas et l'angle d'une rotation et les reporter (`pas_cm`, `rotation_deg`).
5. Fixer les seuils de sécurité (batterie, distance d'arrêt), puis essayer l'exploration en mode **Confirmer chaque plan**.

3.1 JD-Humanoid (XERULITH)

Profil `jd_humanoid`, contrôleur `ezb`, déplacement `jd_actions`. Le JD garde les règles d'origine de Commander, plus strictes que celles des profils génériques :

- calibrage obligatoire (16 servos à 90°) avant tout mouvement ; positions hors course refusées ;
- plus aucun mouvement sous 7,20 V, plus de marche ni d'acrobatie sous 7,45 V (tension médiane de 3 à 7 mesures) ;
- 39 actions classées en geste, déplacement et acrobatie au sol ; les acrobaties exigent la case Acrobaties ;
- tête : panoramique D0, inclinaison D1, centre 90°, balayage d'exploration ±35°.

Port	Articulation	Port	Articulation
D0	Tête — panoramique	D9	Pince droite
D1	Tête — inclinaison	D12	Hanche gauche
D2	Épaule droite	D13	Genou gauche
D3	Épaule gauche	D14	Cheville gauche
D4	Coude gauche	D16	Hanche droite
D5	Avant-bras gauche	D17	Genou droit
D6	Pince gauche	D18	Cheville droite
D7	Coude droit		
D8	Avant-bras droit		

Les directions du pavé appellent les actions `Forward`, `Reverse`, `Left` et `Right` ; un pas « Forward » fait environ 3 cm et une rotation environ 15° (à étalonner). L'annexe A liste la bibliothèque complète.

3.2 Hexapode et quadrupède EZ-B (actions ARC)

Profils `hexa_rover` et `quad_rover`, déplacement `actions`. La marche est une suite de **poses** (Auto Position d'ARC) : XERULITH lit directement un projet ARC (`.EZB`) ou une bibliothèque Commander (`.json`) et joue ses actions sur les servos déclarés dans le profil.

```
{
  "deplacement": {
    "type": "actions",
    "pas_cm": 6,
    "rotation_deg": 20,
    "params": {
      "bibliotheque": "C:\\Robots\\Hexapode.EZB",
      "avant": "Forward",
      "arriere": "Reverse",
```

```
      "gauche": "Left",
      "droite": "Right"
    }
  }
}
```

- Déclarer **tous** les servos de l'hexapode (18 en général) avec leur course réelle : chaque image d'action est vérifiée et refusée si un angle sort de la course déclarée (un servo absent de la liste accepte 1 à 180°).
- Chaque image d'action est envoyée en un seul paquet EZ-B (les servos bougent ensemble), avec la vitesse de l'image ; les pauses d'ARC sont respectées ; STOP interrompt entre deux images.
- Le panneau Actions liste les actions du projet, classées comme celles du JD (le classement suit le nom de l'action) ; les acrobaties restent soumises à la case Acrobaties.
- Avec `fonctions.calibrer` vrai, **Calibrer** place chaque servo au centre de sa course (90° borné à min / max) et devient obligatoire.

3.3 Rover à servos à rotation continue

Profil `rover_roues`, déplacement `servos_continus`. Un servo continu tourne d'autant plus vite que la consigne s'écarte du neutre (90°).

Direction	Roue gauche	Roue droite
avant	neutre + s_g × amplitude	neutre + s_d × amplitude
arrière	neutre − s_g × amplitude	neutre − s_d × amplitude
gauche	neutre − s_g × amplitude	neutre + s_d × amplitude
droite	neutre + s_g × amplitude	neutre − s_d × amplitude

`s_g` et `s_d` valent −1 quand `inverser_gauche` / `inverser_droite` sont vrais (servos montés tête-bêche : en général la roue droite). Le mouvement dure `duree_pas_s × pas`, puis les deux roues reviennent au neutre. Réglage : si le rover tourne au lieu d'avancer, inverser une roue ; s'il dérive au neutre, ajuster `neutre` (89, 91…).

3.4 Rover à pont en H (L298N, TB6612)

Déplacement `hbridge` : quatre sorties numériques EZ-B donnent le sens, deux sorties PWM la vitesse (0 à 100 %).

Direction	IN1 / IN2 (moteur A)	IN3 / IN4 (moteur B)	PWM A / B
avant	1 / 0	1 / 0	vitesse
arrière	0 / 1	0 / 1	vitesse
gauche	0 / 1	1 / 0	vitesse
droite	1 / 0	0 / 1	vitesse
arrêt	0 / 0	0 / 0	0

Si un moteur tourne à l'envers, permuter ses deux fils au bornier ou échanger `a1` et `a2` (`b1` et `b2`) dans le profil.

3.5 Tourelle de caméra (pan / tilt)

Une tourelle se déclare dans **Servos et tête** : les servos dans la liste (avec leur course), puis les cases **Panoramique sur** et **Inclinaison sur**. Une caméra seulement inclinable (GalaxyRVR) n'a que l'inclinaison.

```
{
  "tete": {
    "pan": 7,
```

```
    "tilt": 1,
    "centre_pan": 90,
    "centre_tilt": 90,
    "balayage": 35
  },
  "servos": [
    { "port": 7, "nom": "Servo Rotation", "min": 10, "max": 170 },
    { "port": 1, "nom": "Servo Inclinaison", "min": 10, "max": 170 }
  ]
}
```

- Le pavé Tête du panneau Caméra déplace la tourelle par pas de 8° ; le bouton central la recentre ; **Suivi tête** suit la cible détectée.
- En exploration, `balayage` donne l'amplitude gauche / droite du panorama (0 = une seule vue) ; **Inverser pan** corrige le sens.
- Directive `REGARDER` : gauche / droite à ±balayage, haut +25°, bas −15°, par rapport aux centres.

3.6 ArduRover (rover ArduPilot)

Profil `ardurover`, contrôleur et déplacement `mavlink`. XERULITH agit comme une station sol légère : il lit la télémétrie et envoie des consignes RC (RC override) limitées à la durée d'un pas.

```
{
  "controleur": {
    "type": "mavlink",
    "params": {
      "connexion": "udp:0.0.0.0:14550",
      "baud": 57600,
      "mode_pilotage": "MANUAL"
    }
  },
  "deplacement": {
    "type": "mavlink",
    "pas_cm": 50,
    "rotation_deg": 30,
    "params": {
      "canal_avant": 3,
      "canal_rotation": 1,
      "canal_vertical": 0,
      "pwm_neutre": 1500,
      "amplitude": 200,
      "avance_en_virage": 0,
      "duree_pas_s": 1
    }
  }
}
```

Chaîne de connexion	Cas
`udp:0.0.0.0:14550`	Télémétrie Wi-Fi, MAVProxy ou routeur MAVLink qui envoie vers le PC
`udpout:192.168.4.1:14550`	Le PC envoie vers le véhicule
`tcp:192.168.2.2:5760`	Ordinateur compagnon, SITL
`COM7` (débit `baud`)	Radio de télémétrie USB (SiK 57600 bauds)

- Rover à direction différentielle (skid-steer) : canal 1 = rotation, canal 3 = gaz ; une rotation se fait sur place.
- **RTH** (bouton ⌂ du panneau Déplacement, touche H) : XERULITH passe le véhicule en mode `RTL` d'ArduPilot et vérifie que le mode est confirmé ; le paramètre facultatif `mode_rth` du contrôleur donne un autre mode (par exemple `SMART_RTL`). ArduSub n'a pas de mode RTL : le mode est alors refusé et signalé.
- Direction Ackermann (voiture) : un virage n'est possible qu'en roulant ; `avance_en_virage` (par exemple 80 µs) ajoute un peu de gaz pendant une rotation.

- Le mode `MANUAL` est activé à l'armement ; ArduPilot reste maître (failsafe, geofence, contrôles pré-armement).

3.7 ArduSub (sous-marin)

Profil `sub_rover` : canaux ArduSub **5** avance, **4** lacet (rotation), **3** vertical. `MONTER` et `DESCENDRE` agissent sur le canal vertical ; la profondeur vient de `VFR_HUD` (altitude négative sous l'eau). Le mode de pilotage conseillé est `MANUAL` ou `STABILIZE` ; avec `DEPTH_HOLD`, ArduSub corrige lui-même la profondeur.

```
{
  "deplacement": {
    "type": "mavlink",
    "pas_cm": 30,
    "rotation_deg": 20,
    "params": {
      "canal_avant": 5,
      "canal_rotation": 4,
      "canal_vertical": 3,
      "pwm_neutre": 1500,
      "amplitude": 200,
      "duree_pas_s": 1
    }
  },
  "securite": { "batterie_min": 13, "distance_arret_cm": 0 }
}
```

3.8 Bateau ArduPilot

Profil `boat_rover` : ArduRover avec cadre bateau (`FRAME_CLASS = 2`). Mêmes canaux qu'un rover ; un pas plus long (100 cm) ; pas de distance d'arrêt par défaut. La position GPS et le cap entrent dans chaque observation, ce qui permet des missions de navigation (§ 6.5).

3.9 Carte maison (ESP32, Arduino) en TCP ou série

Contrôleur `tcp` (Wi-Fi) ou `serie` (USB), déplacement `texte`. La carte reçoit une commande texte par ligne et renvoie ses mesures par lignes `CLE=valeur`. Le profil fourni `carte_maison` correspond au croquis ci-dessous : ESP32, pont en H L298N, ultrason HC-SR04, batterie sur un pont diviseur.

```
{
  "controleur": { "type": "tcp", "params": { "adresse": "192.168.4.1:8023" } },
  "deplacement": {
    "type": "texte",
    "pas_cm": 10,
    "rotation_deg": 30,
    "params": {
      "avant": "F {v} {ms}",
      "arriere": "B {v} {ms}",
      "gauche": "L {v} {ms}",
      "droite": "R {v} {ms}",
      "stop": "S",
      "vitesse": 60,
      "duree_pas_ms": 600
    }
  },
  "capteurs": [
    {
      "type": "distance",
      "nom": "Ultrason",
      "source": "texte",
      "params": { "cle": "DIST" }
    },
    {
      "type": "batterie",
      "nom": "Batterie",
      "source": "texte",
      "params": { "cle": "BAT" }
```

```
    }
  ]
}
```

Pour un déplacement de n pas, XERULITH envoie **une** commande où `{v}` est la vitesse, `{ms}` la durée totale (`duree_pas_ms × n`) et `{n}` le nombre de pas, attend cette durée, puis envoie la commande d'arrêt (`S`). STOP envoie aussi `S` immédiatement. Exemple de croquis Arduino pour ESP32 (cœur Arduino-ESP32 3.x) :

```
#include <WiFi.h>
const char* SSID = "MaBox";  const char* PASS = "motdepasse";
WiFiServer server(8023);  WiFiClient cli;
const int IN1 = 26, IN2 = 27, IN3 = 14, IN4 = 12, ENA = 25, ENB = 33;  // L298N
const int TRIG = 5, ECHO = 18, VBAT = 34;      // HC-SR04, pont diviseur 1/3
unsigned long stopAt = 0, nextTx = 0;

void motors(int l, int r) {             // -100..100
  digitalWrite(IN1, l > 0); digitalWrite(IN2, l < 0);
  digitalWrite(IN3, r > 0); digitalWrite(IN4, r < 0);
  analogWrite(ENA, abs(l) * 255 / 100); analogWrite(ENB, abs(r) * 255 / 100);
}
float distanceCm() {
  digitalWrite(TRIG, LOW); delayMicroseconds(2);
  digitalWrite(TRIG, HIGH); delayMicroseconds(10); digitalWrite(TRIG, LOW);
  unsigned long us = pulseIn(ECHO, HIGH, 25000);
  return us ? us / 58.0 : -1;
}
void setup() {
  int pins[] = {IN1, IN2, IN3, IN4, ENA, ENB, TRIG};
  for (int p : pins) pinMode(p, OUTPUT);
  pinMode(ECHO, INPUT);
  WiFi.begin(SSID, PASS); while (WiFi.status() != WL_CONNECTED) delay(200);
  server.begin();
}
void loop() {
  if (!cli || !cli.connected()) { cli = server.available(); }
  if (cli && cli.available()) {
    String l = cli.readStringUntil('\n'); l.trim();
    char c = l.charAt(0); int v = 60, ms = 600;
    sscanf(l.c_str() + 1, "%d %d", &v, &ms);
    if (c == 'F') motors(v, v); else if (c == 'B') motors(-v, -v);
    else if (c == 'L') motors(-v, v); else if (c == 'R') motors(v, -v);
    else { motors(0, 0); ms = 0; }
    stopAt = ms ? millis() + ms : 0;
  }
  if (stopAt && millis() > stopAt) { motors(0, 0); stopAt = 0; }   // arrêt local
  if (cli && cli.connected() && millis() > nextTx) {
    float vbat = analogReadMilliVolts(VBAT) * 3.0 / 1000.0;
    cli.printf("DIST=%.1f BAT=%.2f\n", distanceCm(), vbat);
    nextTx = millis() + 200;
  }
}
```

- La carte doit arrêter seule ses moteurs à la fin de la durée reçue : c'est la sécurité en cas de coupure Wi-Fi.
- Autres mesures possibles : `GPS=48.5436,3.0036`, `CAP=270`, `PROF=1.2`, `IR=1`, `A0=512` — chaque clé est affectée à un capteur dans le profil. Les trames NMEA (`$GPGGA…`, `$GPRMC…`) envoyées telles quelles sont aussi décodées.
- En série (`serie`), même protocole sur le port USB de la carte (115 200 bauds par défaut).

3.10 SunFounder GalaxyRVR

Profil `galaxy_rvr`, contrôleur et déplacement `galaxyrvr`. Le rover garde son firmware d'origine : XERULITH dialogue avec son ESP32-CAM comme l'application SunFounder Controller (protocole détaillé au § 4.4).

```
{
```

```
  "controleur": {
    "type": "galaxyrvr",
    "params": { "adresse": "192.168.4.1", "protocole": "auto" }
  },
  "deplacement": {
    "type": "galaxyrvr",
    "pas_cm": 8,
    "rotation_deg": 25,
    "params": {
      "vitesse": 60,
      "vitesse_rotation": 70,
      "duree_pas_s": 0.5,
      "duree_rotation_s": 0.35
    }
  },
  "tete": {
    "pan": null,
    "tilt": 6,
    "centre_pan": 90,
    "centre_tilt": 90,
    "balayage": 0
  },
  "capteurs": [
    {
      "type": "obstacle",
      "nom": "Infrarouge gauche",
      "source": "galaxyrvr",
      "params": { "cote": "gauche" }
    }
  ],
  "securite": { "batterie_min": 6.6, "distance_arret_cm": 20 }
}
```

- Réseau : Wi-Fi « GalaxyRVR » (mot de passe 12345678, adresse 192.168.4.1) ou mode client sur la box ; fermer l'application SunFounder.
- Firmware 2.x : port 30102, trames binaires ; firmware 1.x : port 8765, JSON. `protocole` vaut `auto`, `v2` ou `v1` ; un port écrit dans l'adresse (`192.168.4.1:8765`) est utilisé tel quel.
- Caméra : flux MJPEG `http://<ip>:9000/mjpg` (ou l'adresse annoncée par l'ESP32) ; inclinaison 0 à 140°.
- Étalonnage : mesurer la distance d'un pas à 60 % et l'angle d'une rotation à 70 % sur votre sol, puis reporter `pas_cm` et `rotation_deg`.

3.11 Simulateur

Cocher **Simulateur** avant de connecter : le JD utilise le simulateur de Commander ; tout autre profil utilise un rover virtuel dans une pièce de 4 m × 3 m avec trois obstacles (caisse rouge, carton bleu, plante verte). Distance, infrarouges, GPS (autour de la position de départ du capteur GPS sim), cap, profondeur et batterie sont calculés ; la caméra de synthèse montre la pièce vue de dessus et le champ du télémètre. Le simulateur sert à mettre au point un profil, une mission ou un script sans risque.

3.12 Drone Arma 3 (Pythia)

Profil `drone_arma3`, type `drone`, contrôleur et déplacement `arma3`. Le « robot » est un drone AR-2 Darter (`B_UAV_01_F`) du simulateur Arma 3, piloté par le pont `@XerulithBridge` (§ 4.6). La caméra est une capture de la fenêtre du jeu (source `ecran`) ; toutes les autres mesures viennent de la télémétrie du pont (source `arma3`).

```
{
  "type": "drone",
  "controleur": {
    "type": "arma3",
    "params": {
      "adresse": "127.0.0.1:47800",
      "origine_lat": 48.5436,
      "origine_lon": 3.0036
```

```
    }
  },
  "deplacement": {
    "type": "arma3",
    "pas_cm": 300,
    "rotation_deg": 45,
    "params": {
      "vitesse_ms": 3,
      "vitesse_verticale_ms": 1.5,
      "rotation_degs": 45,
      "duree_pas_s": 1,
      "vitesse_transit_ms": 0,
      "rayon_max_m": 6000,
      "hauteur_max_m": 150
    }
  },
  "tete": {
    "pan": 0,
    "tilt": 0,
    "centre_pan": 0,
    "centre_tilt": -15,
    "balayage": 40
  },
  "capteurs": [
    {
      "type": "camera",
      "nom": "Caméra du drone",
      "source": "ecran",
      "params": {
        "fenetre": "Arma 3",
        "ips": 15,
        "largeur": 960,
        "incrustation": true
      }
    },
    {
      "type": "analogique",
      "nom": "Radiogonio 121,5 MHz — niveau",
      "source": "arma3",
      "params": { "mesure": "balise_dbm", "unite": "dBm" }
    }
  ],
  "securite": { "batterie_min": 0, "distance_arret_cm": 500 }
}
```

Commandes et vol physique

En mode de vol **physique** (défaut du script de mission), le drone vole avec le modèle de vol d'Arma ; son IA pilote, en comportement `CARELESS` et combat `BLUE`, sert de pilote automatique. XERULITH lui donne des **consignes** :

Commande XERULITH	Ordre du pont	Traduction dans le script SQF
avant / arrière (n pas)	`vel` : avant ±v, durée n × `duree_pas_s`	point cible à v × durée dans le repère du drone ; `limitSpeed` v ; `doMove` ; cap d'origine repris à l'arrivée
gauche / droite	`vel` : lacet ±`rotation_degs`	cap cible = cap − lacet × durée (lacet positif = gauche), tenu en stationnaire
monter / descendre	`vel` : haut ±`vitesse_verticale_ms`	hauteur de vol `flyInHeight` ± haut × durée (2 à 500 m)
ALLER / VOLER / HAUTEUR	`goto` : x, y (carte), h, v	point de la carte, `limitSpeed` v (sans limite si v = 0), `forceSpeed -1`, `flyInHeight` h
STOP	`stop`	point d'arrêt = position + vitesse × 1 s : le drone freine avec son inertie

Le script ajoute trois boucles de commande, exécutées à chaque image (`EachFrame`) :

Boucle	Loi de commande	Rôle
Maintien de cap	rotation des vecteurs `vectorDir` / `vectorUp` autour de la verticale, au plus `xer_yawrate` × dt	tient le cap en stationnaire sans toucher à l'inclinaison calculée par Arma
Maintien de hauteur	vz consigne = 0,6 × (h cible − h), bornée à ±vz max ; az = 1,5 × (vz consigne − vz), bornée à ±3 m/s²	supprime le « pompage » de l'IA autour de sa hauteur de vol
Maintien de position	a = 0,4 × (cible − position) − 1,2 × v (horizontal), bornée à 2 m/s²	stationnaire de type GPS quand l'IA ne vole pas vers un point ; termine les trajets que l'IA arrête à quelques mètres

- **Arrivée** : pas élémentaire à moins de 2,5 m (ou IA prête) et vitesse < 4 km/h après 1 s ; trajet `goto` à moins de 8 m et vitesse < 8 km/h après 2 s ; délai maximal 2 × durée + 10 s (pas) ou 2 × distance / v + 40 s (trajet). Le drapeau `occupe` de la télémétrie vaut 1 tant que le drone se déplace, tourne ou change de hauteur (|vz| > 0,6 m/s).
- **Attente côté XERULITH** : après un pas, XERULITH attend 0,6 s puis la fin de `occupe` (au plus 4 × durée + 6 s) ; un trajet attend au plus 2 × distance / v + 45 s (v = 12 m/s si vitesse maximale). Un STOP pendant l'attente envoie `stop`.
- **Mode arcade** : `setVelocityModelSpace` et `setVectorDirAndUp` à chaque image (drone à plat), arrêt net à la fin du pas (`fin`) ; `goto` n'y est pas disponible.
- **Vue** `camera` (défaut) : caméra `camCreate` attachée au véhicule (sous le nez du drone, [0 ; 1,1 ; 0,9] m sur le rover), orientée par `setVectorDirAndUp` dans le repère du véhicule, sans l'interface d'Arma (chat masqué par `showChat false` après 8 s). Vue `hud` : `player connectTerminalToUAV`, `player remoteControl (gunner _uav)` puis `switchCamera "GUNNER"`, tourelle pointée par `lockCameraTo` sur un point à 300 m (azimut = cap − pan, élévation = tilt), rafraîchi à 10 Hz. Vue `conducteur` (rover) : `switchCamera "INTERNAL"` sans prise de contrôle ; les vues tournées et le FLIR passent sur une caméra auxiliaire.
- **Position** : x (est) et y (nord) de la carte, en mètres, convertis en latitude / longitude autour de l'origine du profil : lat = lat0 + y / 111 320, lon = lon0 + x / (111 320 × cos lat0).

Capture vidéo et caméra thermique

HUD de XERULITH (`xerulith/hud.py`) : `hud_state(robot)` relève la télémétrie du pilote du profil (cap, vitesse, hauteur, altitude, tangage, roulis, profondeur, télémètre, carburant, batterie, dégâts, radiogoniomètre, position depuis le départ, cible du trajet en cours `Arma3Backend.target`, objets vus `seen_recent()`), et `paint_hud(painter, rect, état)` dessine dans `CameraView.paintEvent` : bande de cap ±60° avec repères (H départ, cible, #n objets, 121,5 balise, CAM direction de la caméra), horizon artificiel, colonnes de mesures, position. Il est dessiné à l'affichage seulement (case **HUD** du panneau Caméra) : les images transmises au pilote IA et aux détecteurs restent sans HUD. Il est indépendant de la source vidéo : seules les mesures fournies par le robot apparaissent.

La caméra `ecran` capture la zone client de la fenêtre du simulateur, 15 images/s, redimensionnée à 960 pixels de large. XERULITH retient la fenêtre du **programme** `arma3_x64.exe` de préférence à une fenêtre dont seul le titre contient « Arma 3 » (dossier ouvert dans l'Explorateur, lanceur) ; il ne capture jamais sa propre fenêtre. Le journal indique la fenêtre retenue. Le bandeau de télémétrie (2 ou 3 lignes) est ensuite incrusté en bas de l'image.

En caméra thermique (`FLIR: on`, blanc = chaud), XERULITH s'appuie sur le **détecteur thermique simulé** du pont (champ `chaud`, pont 1.7) : la caméra est tournée vers chaque vue (gauche, centre, droite) et le détecteur est relu. Foyer dans l'image : la caméra reste pointée et le pont renvoie la position au sol du point (`pointer`, puis `screenToWorld`), convertie en E / N, latitude et longitude (degrés décimaux et degrés, minutes, secondes) pour l'observation du pilote. Foyer visible du drone mais hors de l'image : **pointage automatique** — pan = cap − relèvement (positif à gauche), inclinaison = −atan(hauteur / distance) −

inclinaison centrale du profil, bornée à −89° ; après 0,9 s le détecteur est relu. Foyer caché par le relief : seul le panache est signalé (relèvement, distance, directive `VOLER`). Avec un ancien pont sans champ `chaud`, l'image thermique est analysée : pixels presque blancs (seuil adaptatif), nettoyage morphologique qui efface le texte et le réticule du HUD, taches de 0,004 % à 20 % de l'image.

Point chaud enregistré (position au sol mesurée) : tonalité d'alerte sur le PC (3 × 880 / 1320 Hz), photo de la vue tournée vers le point chaud, annotée (cercle, bandeau : n°, heure, cycle, E / N, DMS, degrés décimaux, distance), `point_chaud_NN.jpg` et une ligne de `points_chauds.csv` (séparateur `;`) dans le dossier de la mission ; un point à moins de 60 m d'un point enregistré n'est pas répété ; le rapport ajoute la section « Points chauds détectés ». La caméra repasse en vision normale sur `ALLER: depart` et en fin de mission.

3.13 Rover UGV Arma 3 (Pythia)

Profil `ugv_arma3`, type `rover`, contrôleur et déplacement `arma3`. Le « robot » est un rover terrestre sans équipage **UGV Stomper** (`B_UGV_01_F`), créé par `init.sqf` avec `_vehicule = "ugv"` (dossier `mission_rover`) ou désigné par le nom de variable `xer_vehicule` dans l'éditeur. Le même pont, le même script et le même pilote (`Arma3Backend`) servent : le pont détecte le véhicule terrestre (`isKindOf "LandVehicle"`, champ `sol` = 1) et passe en conduite.

```
{
  "type": "rover",
  "deplacement": {
    "type": "arma3",
    "pas_cm": 300,
    "rotation_deg": 45,
    "params": {
      "vitesse_ms": 2,
      "rotation_degs": 30,
      "duree_pas_s": 1.5,
      "vitesse_transit_ms": 4,
      "rayon_max_m": 3000
    }
  },
  "tete": {
    "pan": 0,
    "tilt": 0,
    "centre_pan": 0,
    "centre_tilt": -5,
    "balayage": 60
  },
  "capteurs": [
    {
      "type": "analogique",
      "nom": "Tangage",
      "source": "arma3",
      "params": { "mesure": "tangage", "unite": "°" }
    },
    {
      "type": "analogique",
      "nom": "Roulis",
      "source": "arma3",
      "params": { "mesure": "roulis", "unite": "°" }
    }
  ],
  "securite": { "batterie_min": 0, "distance_arret_cm": 300 }
}
```

Commande XERULITH	Ordre du pont	Traduction dans le script SQF (rover)
avant / arrière, gauche / droite	`vel`	l'IA lâche son trajet (`doMove` sur place) ; à chaque image pendant la durée du pas : `setVelocityModelSpace [droite, avant, vz]` (vz laissée à la physique) et rotation sur place des

Commande XERULITH	Ordre du pont	Traduction dans le script SQF (rover)
		vecteurs `vectorDir` / `vectorUp`
monter / descendre, HAUTEUR	—	refusés par XERULITH
ALLER / ROULER / SUIVANT	`goto` : x, y, v	`limitSpeed` v, `doMove` : l'**IA conductrice** (comportement `AWARE`, trajet direct en tout-terrain plutôt que par les routes, combat `BLUE`) choisit le chemin et contourne les obstacles
STOP	`stop`	`doMove` sur la position actuelle (vitesse annulée en conduite directe)

- **Conduite directe** (secours) : si l'IA conductrice reste immobile 10 s au début d'un trajet (4 s en vue `conducteur`) ou se déclare prête loin du point après 6 s, le pont conduit lui-même à chaque image : rotation des vecteurs vers le relèvement du point (35 °/s au plus), `setVelocityModelSpace [0, v, vz]` avec v = min(vitesse demandée ou 8 m/s, 14 m/s, distance / 2,5), au moins 1,5 m/s, 1 m/s si l'écart de cap dépasse 40° ; arrivée à 12 m ; blocage après 12 s d'immobilité.

- **Arrivée** : à moins de 12 m du point, vitesse < 3 km/h (quelconque en conduite directe), après 2 s. **Blocage** (`bloque` = 1) : en conduite directe, immobile (< 1 km/h) depuis 12 s, ou délai 2 × distance / v + 60 s dépassé (v = 6 m/s si vitesse maximale). `occupe` vaut 1 pendant un trajet, un pas, ou tant que la vitesse dépasse 1,5 km/h.
- **Côté XERULITH** : un trajet attend au plus 2 × distance / v + 75 s ; il est déclaré « bloqué » si `bloque` = 1 ou si le rover s'arrête à plus de 25 m du point. L'explorateur recule alors de 15 m et essaie un point de contournement décalé de 100 m à gauche (perpendiculaire au trajet, 40 m en avant), puis à droite, en évitant la mer, avant de rendre la main au pilote.
- **Télémètre** : `lineIntersectsSurfaces` depuis 1,6 m devant le centre et 0,6 m de haut, sur 150 m (le terrain qui monte devant compte comme obstacle). **Caméra** : tourelle du gunner (`hud`), ou caméra scriptée à [0 ; 1,1 ; 0,9] m (`camera`, FLIR).
- **Assiette** : `tangage` et `roulis` en degrés ; l'observation affiche une **ALERTE PENTE** au-delà de 20° de tangage ou 15° de roulis, et le rapport garde la plus forte inclinaison rencontrée.
- **Bandeau** : VIT, CAP, ALT, TANGAGE, ROULIS ; télémètre, carburant, dégâts, « ROVER », « EN MOUVEMENT », « BLOQUÉ » ; nombre d'objets en vue du détecteur.

4. Protocoles de communication

4.1 EZ-B v4

TCP port **23** (commandes) et **24** (caméra JPEG). À la connexion, XERULITH envoie l'octet `0x55` ; une EZ-B v4 répond `0x2A`. Une seule connexion est acceptée sur le port 23 : ARC doit être fermé.

Commande	Octets	Remarque
Position d'un servo	`0xAC + port`, angle	angle 1-180 ; 0 relâche le servo
Vitesse d'un servo	`0x27 + port`, vitesse	0 (rapide) à 20 (lent)
Sortie numérique basse / haute	`0x64 + port` / `0x7C + port`	pont en H
Lecture numérique	`0x94 + port`	réponse 1 octet
Lecture ADC	`0xC4 + port`	ports A0-A7
PWM	`0x0F + port`, rapport 0-100	pont en H
I2C écriture / lecture	`0x0A` adr, n, données / `0x0B` adr, n	SRF02/08/10 : écrire 0x00 0x51 (mesure en cm), attendre 70 ms, lire 2 octets au registre 0x02
Utilitaires	`0x04` + sous-commande	batterie 0x02, température 0x03, UART init 0x05/0x09/0x0D, écriture, disponible, lecture

Les servos d'un profil non JD sont écrits directement (`0xAC + port`) après vérification de la course **déclarée dans le profil** : les limites propres au JD ne s'appliquent qu'au profil JD. Une image d'action est envoyée en un seul paquet pour que les servos bougent ensemble.

4.2 MAVLink (ArduPilot)

Sens	Message	Usage
reçu	HEARTBEAT	liaison vivante (5 s), armement (`base_mode` & 128), mode de vol
reçu	GLOBAL_POSITION_INT	latitude, longitude, altitude, cap
reçu	GPS_RAW_INT	type de fix, nombre de satellites
reçu	VFR_HUD	cap, vitesse sol, profondeur ArduSub (altitude négative)
reçu	SYS_STATUS	tension et pourcentage de batterie
reçu	DISTANCE_SENSOR	télémètres ; orientation 0 (avant) en priorité
émis	REQUEST_DATA_STREAM	tous les flux à 4 Hz
émis	COMMAND_LONG (ARM_DISARM)	armement / désarmement
émis	SET_MODE	mode du profil à l'armement
émis	RC_CHANNELS_OVERRIDE	10 Hz pendant un pas ; 65535 = canal ignoré, 0 = rendu à la radiocommande

Séquence d'un pas : consignes répétées à 10 Hz pendant `duree_pas_s × n`, puis trois envois au neutre (`pwm_neutre`), puis rendu des canaux à la radiocommande. Le pas s'interrompt si STOP est demandé ou si le véhicule est désarmé.

4.3 Carte maison

Sens	Format	Exemple
vers la carte	une commande par ligne, terminée par `\n`	`F 60 1200`, `S`
vers le PC	`CLE=valeur` ou `CLE:valeur`, séparés par espaces ou `;`	`DIST=42 BAT=7.62;IR=0`
vers le PC	couple `lat,lon`	`GPS=48.5436,3.0036`
vers le PC	trames NMEA GGA et RMC (somme de contrôle vérifiée)	`$GPGGA,123519,4807.038,N,…*47`

4.4 SunFounder GalaxyRVR

L'ESP32-CAM ouvre une WebSocket et relaie les messages vers l'Arduino par sa liaison série (`WS+` texte, `WSB+` binaire). À la connexion, il envoie une fiche JSON (`Name`, `Type`, `video`…). **Sans données pendant 3 s**, il envoie `[APPSTOP]` à l'Arduino, qui s'arrête et cesse d'envoyer ses mesures : XERULITH renvoie donc l'état complet toutes les 0,3 s.

Firmware 2.x — port 30102, trames binaires

Octet	Contenu
0	`0xA0` début
1	n = nombre d'octets de données
2	somme de contrôle : OU exclusif des n octets de données
3 … 2+n	entités
3+n	`0xA1` fin

Entité	Sens	Données
`0x01`	PC → rover	moteur gauche, moteur droit (entiers signés −100…100)
`0x02`	PC → rover	LED RGB : r, v, b
`0x03`	PC → rover	angle du servo de caméra (0…140)
`0x04`	PC → rover	lampe de la caméra : 1 / 0
`0x05`	PC → rover	modes évitement / suivi d'obstacle (non utilisés par XERULITH)
`0x81`	rover → PC	ultrason en mm sur 2 octets (poids fort d'abord)
`0x82`	rover → PC	infrarouges : bit 1 gauche, bit 0 droit (1 = obstacle)
`0x83`	rover → PC	batterie : (tension − 6 V) × 100

Trame type envoyée par XERULITH pour avancer à 60 % avec la caméra à 90° : `A0 05 58 01 3C 3C 03 5A A1`.

Firmware 1.x — port 8765, JSON

XERULITH envoie `{"K": gauche, "Q": droite, "D": angle}` (zones K et Q de l'application : moteurs ; zone D : servo). Le rover répond `{"O": distance_cm, "N": ir_gauche, "P": ir_droit, "BV": tension}` ; `0` vaut −1 hors de portée.

4.5 Flux vidéo et GPS

- Caméras réseau et USB : lues par OpenCV dans un fil, environ 10 images/s, réencodées en JPEG ; reconnexion automatique après 3 s.

- GPS NMEA : sur une UART de l'EZ-B (interrogée toutes les 0,5 s), sur un port série du PC (pyserial) ou en TCP (port 10110 par défaut, reconnexion automatique).

4.6 Pont Arma 3 (Pythia)

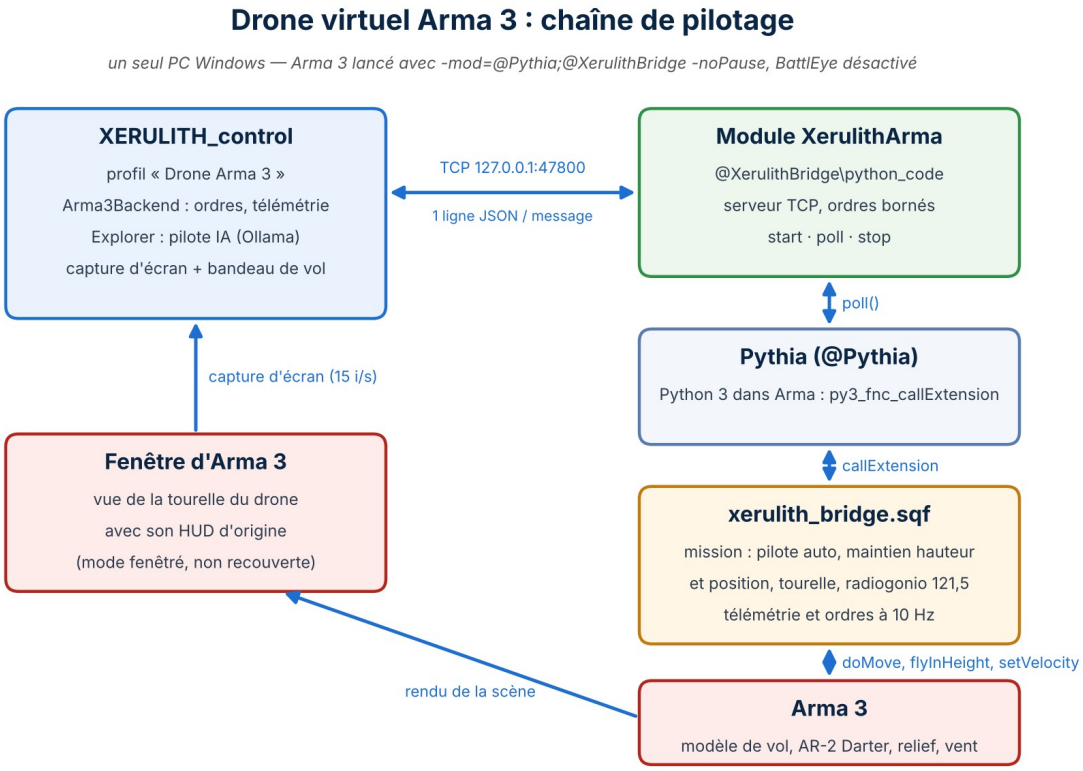


Figure 1 — Chaîne de pilotage du drone Arma 3.

Le pont relie XERULITH au script de mission par TCP. Le serveur tourne **dans Arma**, dans le module Python `XerulithArma` exécuté par le mod Pythia ; le script `xerulith_bridge.sqf` l'appelle dix fois par seconde. Le module n'utilise que la bibliothèque standard de Python.

Chargement par Pythia

- Appel SQF : `["XerulithArma.poll", [_tel]] call py3_fnc_callExtension` ; le nom de module vient du fichier `python_code\$PYTHIA$` (contenu : `XerulithArma`).
- Pythia ne cherche les dossiers `$PYTHIA$` que dans les mods dont Arma a **ouvert un fichier .pbo** : il parcourt les .pbo chargés et examine les sous-dossiers de leur mod. D'où l'addon minimal `addons\xerulith_bridge.pbo` (une classe `CfgPatches Xerulith_Bridge`), sans lequel l'appel échoue (« module not recognized »).

Messages de XERULITH vers Arma

Une ligne JSON par message, terminée par `\n`. Seuls ces ordres existent ; toute valeur est bornée par le module avant d'atteindre le SQF :

Message	Bornes	Ordre SQF
`{"cmd":"hello"}`	—	aucun (le serveur envoie `{"t":"hello","pont":"XerulithArma","version":"1.7"}` à la connexion)
`{"cmd":"vel","avant","droite","haut","lacet","duree"}`	±60, ±60, ±20 m/s, ±120 °/s, 0 à 30 s	`["vel", avant, droite, haut, lacet, duree]`
`{"cmd":"fin"}`	—	`["fin"]` : fin d'un pas (arrêt en arcade, ignoré en physique)

Message	Bornes	Ordre SQF
`{"cmd":"stop"}`	—	`["stop"]` ; retire les ordres en attente sauf `cam`
`{"cmd":"cam","pan","tilt"}`	±180°, ±90°	`["cam", pan, tilt]` (pan positif = gauche)
`{"cmd":"goto","x","y","h","v"}`	x, y 0 à 200 000 m (−1 = inchangé) ; h 2 à 500 m (−1) ; v 0 à 100 m/s (0 = maximale)	`["goto", x, y, h, v]`
`{"cmd":"balise","rayon_min","rayon_max"}`	50 à 10 000 m	`["balise", rmin, rmax]` (une seule balise par mission)
`{"cmd":"flir","actif"}`	vrai / faux	`["flir", 1|0]` : `setCamUseTI` (blanc = chaud) ; en vue tourelle, bascule sur une caméra scriptée fixée sous le nez du drone (la thermique de la tourelle n'est pas pilotable par script), retour à la vue tourelle à l'arrêt
`{"cmd":"pointer","u","v"}`	0 à 1	`["pointer", u, v]` : point de l'image (u vers la droite, v vers le bas) dont la télémétrie renvoie la position au sol
`{"cmd":"sonde","x0","y0","dx","nx","ny"}`	pas 10 à 1000 m, 200 × 200 nœuds au plus	`["sonde", …]` : carte terre / mer de la grille, réponse dans le champ `sonde`
`{"cmd":"feu","x0","y0","x1","y1","proba"}`	rectangle de la carte, 0 à 1	`["feu", x0, y0, x1, y1, proba]` : départ de feu au hasard dans le rectangle (un seul par mission)
`{"cmd":"sites","x0","y0","x1","y1","n"}`	rectangle de la carte, 1 à 10 objets	`["sites", x0, y0, x1, y1, n]` : objets à identifier placés au hasard (exercice rover, une fois par mission)

Un client perdu ajoute un `stop` à la file : le drone s'arrête si XERULITH se ferme ou perd la liaison.

Télémétrie d'Arma vers XERULITH

Champ	Unité	Source SQF
`monde`	nom de carte	`worldName`
`x`, `y`, `z_asl`	m	`getPosASL`
`hauteur`	m sol	`(getPos _uav) select 2`
`cap`	°	`getDir`
`vitesse`	km/h	`speed`
`carburant`, `degats`	0 à 1	`fuel`, `damage`
`distance`	m, −1 hors portée	`lineIntersectsSurfaces` sur 150 m devant le drone
`temps`, `drone`	s, classe	`time`, `typeOf`
`mode`, `occupe`	texte, 0 / 1	mode de vol ; déplacement en cours
`balise_cap`, `balise_dbm`, `balise_dist`	°, dBm, m	radiogoniomètre (−1 / −999 sans balise) ; `balise_dist` = distance vraie, réservée à l'évaluation

Champ	Unité	Source SQF
`pt_x`, `pt_y`	m, −1	`screenToWorld` du point demandé par `pointer` (−1 : ciel ou plus de 6 km)
`flir`	0 / 1	caméra thermique active
`feu_x`, `feu_y`	m	exercice incendie : −1 aucun, −2 exercice sans feu, sinon position vraie (évaluation seulement)
`sonde`	texte	vide, ou une fois après l'ordre `sonde` : un caractère par nœud, ligne par ligne depuis le sud-ouest (`1` = mer)
`sol`	0 / 1	1 = véhicule terrestre (`isKindOf "LandVehicle"`)
`tangage`, `roulis`	°	assiette : `asin` de la composante verticale de `vectorDir` (+ = nez en haut) et du vecteur droit (+ = penché à droite)
`bloque`	0 / 1	rover : dernier trajet abandonné (immobilisé loin du point)
`sites`	texte	objets placés `n;nom;x;y|…` (vérité terrain, évaluation seulement)
`vus`	texte	détecteur d'objets simulé : `n;distance;relèvement|…` des objets vus par la caméra
`chaud`	texte	détecteur thermique simulé (FLIR actif, feu à moins de 3 km) : `u;v;distance;relèvement;foyer` ; u, v position du foyer dans l'image (0 à 1, −1 hors de l'image ou caché), foyer = 1 si le foyer est en vue directe du drone (`terrainIntersectASL`), sinon seul le panache (40 m au-dessus) est vu ; vide sans feu

```
→ {"cmd": "goto", "x": 14520.0, "y": 16310.0, "h": -1, "v": 0}
← {"t": "tel", "monde": "Altis", "x": 14231.4, "y": 16012.8, "z_asl": 83.2,
"hauteur": 60.1,
   "cap": 44.0, "vitesse": 52.3, "carburant": 0.97, "degats": 0, "distance": -1,
"temps": 412.6,
   "drone": "B_UAV_01_F", "mode": "physique", "occupe": 1, "balise_cap": -1,
"balise_dbm": -999, "balise_dist": -1}
```

Balise 121,5 MHz et radiogoniomètre

- **Placement** (`xer_fnc_balise`) : jusqu'à 80 tirages d'un point à une distance aléatoire entre `rayon_min` et `rayon_max` et un azimut aléatoire, retenu s'il est hors de l'eau (`surfaceIsWater`) et plat (`isFlatEmpty`) ; épave `C_Heli_Light_01_civil_F` détruite sans effets (`setDamage [1, false]`), blessé `C_man_1` inconscient à 7 m, antenne à 1,5 m du sol ; repère caché `xer_balise_mk` (alpha 0).
- **Niveau** : P = 17 − (20 $\log_{10}$ d + 74,1) − 22 × [relief entre drone et balise] + bruit uniforme ±3 dB, plafonné à −15 dBm ; d en km (distance 3D), relief testé par `terrainIntersectASL`. Signal exploitable au-dessus de −112 dBm.
- **Relèvement** : azimut vrai `dx atan2 dy` + bruit gaussien `random [-s, 0, s]`, s = 10° en vue directe, 35° derrière le relief, 70° à moins de 40 m (à l'aplomb).

Départ de feu (surveillance incendie)

- **Placement** (`xer_fnc_feu`) : point au hasard dans le rectangle, hors de l'eau (en formation, XERULITH donne un rectangle de 80 m autour d'un point situé entre deux points de passage d'une même passe du plan de vol : le feu est devant le drone, à moins de 500 m) ; foyer `test_EmptyObjectForFireBig` (flammes et fumée) et véhicule `C_Offroad_01_F` détruit en feu à 6

m, pour une signature thermique nette ; repère caché `xer_feu_mk`. XERULITH demande le placement avec une probabilité de 1 : il y a toujours un feu.

- **Carte terre / mer** (`sonde`) : grille de la zone + 500 m (maille 100 m), `surfaceIsWater` sur chaque nœud, renvoyée une fois dans le champ `sonde` de la télémétrie (`1` = mer). XERULITH refuse un ALLER ou un VOLER vers la mer et contourne la mer par la terre (recherche A* sur la grille, mer élargie d'une maille, trajet lissé).
- **Plan de vol géré par XERULITH** (directive `SUIVANT`) : XERULITH quadrille lui-même la zone écrite dans la mission (passes nord-sud alternées réparties sur toute la largeur ; espacement « espacées de 1000 m » et pas « tous les 500 m » lus dans le texte), retire les points en mer et garde l'indice du prochain point. L'observation donne « k/N points faits » et le prochain point ; `SUIVANT` le rejoint (par la terre), et un point refusé ou bloqué est sauté. Le modèle n'a plus de géométrie à faire : réponses courtes, départ immédiat, aucune boucle sur les points en mer.

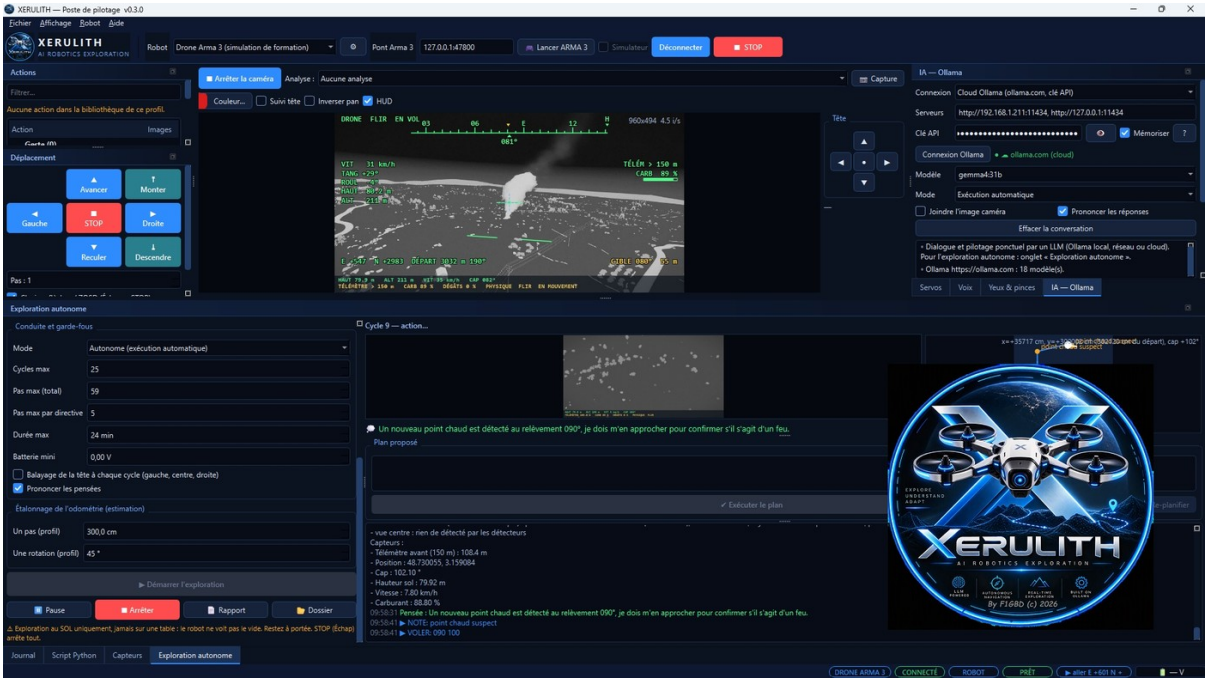


Figure 2 — Caméra thermique : panache du départ de feu détecté au relèvement 090°, le pilote s'en approche.



Figure 3 — Confirmation en vision normale : foyer et fumée sous le drone, HUD de XERULITH.

- **Évaluation** : le rapport compare les notes du pilote qui parlent de feu (position mesurée du point chaud, ou du drone à défaut) à la position vraie ; le feu est localisé si l'écart est inférieur à 150 m.

Objets à identifier (exploration rover)

- **Placement** (`xer_fnc_sites`) : n objets au hasard dans le rectangle, hors de l'eau et sur un sol peu pentu (`isFlatEmpty`), choisis parmi des classes vanilla vérifiées par `isClass`

(`Land_CargoBox_V1_F`, `Land_Wreck_Offroad_F`, `Land_TentDome_F`, `Land_MetalBarrel_F`, `Land_DataTerminal_01_F`, `Land_Wreck_Car2_F`, `Land_Pallets_stack_F`, `Land_PortableLight_double_F`) ; repères cachés `xer_site_mk_<n>`.

- **Détecteur d'objets** (`xer_fnc_vus`, 10 Hz) : objet à moins de 80 m, à moins de 50° de l'axe de la caméra (cap − pan) ou à moins de 12 m, et visible (`checkVisibility` > 0,2 depuis la caméra) ; XERULITH convertit distance et relèvement en position estimée et garde la dernière détection de chaque objet.

Exploitation par XERULITH

- **RTH** (bouton ⌂ du panneau Déplacement, touche H) : une mission en cours est arrêtée, puis `goto` vers le point de départ (`Arma3Backend.home`, fixé à la première télémétrie puis au lancement de chaque mission) à la hauteur courante.
- XERULITH en tire `balise_cap`, `balise_rel` (écart avec le cap du drone, positif à droite) et `balise_dbm` ; il garde la plus courte distance vraie (`beacon_min`) pour le rapport.

5. Les modèles d'IA

5.1 Connexion à Ollama

Connexion	Comportement
Réseau local	Serveurs de la liste essayés dans l'ordre (par défaut http://192.168.1.211:11434, http://127.0.0.1:11434) ; test par `/api/version`
Cloud Ollama	`https://ollama.com`, clé API obligatoire (ollama.com → Settings → Keys) ; test par `/api/tags`, qui valide aussi la clé
Automatique	Réseau local d'abord, puis le cloud si aucun serveur ne répond

- La clé n'est envoyée qu'à ollama.com, jamais à un serveur local. Elle n'est enregistrée que si la case est cochée, chiffrée par DPAPI Windows (liée au compte utilisateur).
- Erreurs traduites : clé refusée (HTTP 401/403), quota atteint (HTTP 429), serveur injoignable, modèle absent.
- Un serveur réseau doit écouter sur toutes les interfaces : `OLLAMA_HOST=0.0.0.0`, port 11434 ouvert dans le pare-feu.

5.2 Rôles des modèles

Rôle	Défaut local	Défaut cloud	Exigence
Décision (pilote)	`gemma3:12b`	`gemma4:31b`	suivre un format strict, raisonner en français ; 8 à 30 milliards de paramètres
Vision (description)	`qwen3-vl:8b`	`gemma4:31b`	modèle multimodal (images) ; décrit le panorama en 3 phrases
Dialogue (onglet IA)	au choix	au choix	modèle vision si « Joindre l'image caméra »

En local, 16 Go de mémoire graphique permettent de garder `gemma3:12b` et `qwen3-vl:8b` chargés ensemble (`keep_alive` = 15 min) ; avec moins, Ollama recharge les modèles à chaque cycle et l'exploration ralentit. Les modèles d'embedding (`bge`, `minilm`, `embed`…) sont refusés pour la décision.

```
ollama pull gemma3:12b
ollama pull qwen3-vl:8b
ollama ps          # modèles chargés et mémoire utilisée
```

5.3 Paramètres des requêtes

Paramètre	Valeur	Effet
`temperature`	0.4	faible : réponses régulières et respect du format
`num_ctx`	8192	contexte : invite système, observation, mémoire et historique
`num_predict`	400 (décision), 220 (vision), 900 (rapport)	longueur maximale de la réponse
`think`	false	modèles « thinking » (qwen3…) : pas de raisonnement caché ; repli automatique si le serveur refuse l'option
`keep_alive`	15m	modèle gardé en mémoire entre deux cycles
délai	240 s par requête d'exploration	au-delà, le cycle est compté en erreur et recommence

Les balises `<think>…</think>` éventuelles sont retirées de la réponse avant analyse.

5.4 Invite système du pilote

L'invite système dépend du profil. Pour le JD, elle décrit le bipède de 35 cm, sa fragilité et ses gestes. Pour les autres robots, elle est construite à partir du profil :

- la **description** du profil, son type et la liste de ses capteurs ;
- la distance d'un pas et l'angle d'une rotation (`pas_cm`, `rotation_deg`, ou l'étalonnage du panneau) ;
- les **seules** directives permises pour ce robot : `MONTER` / `DESCENDRE` pour un sous-marin ou un drone, `ALLER`, `VOLER`, `HAUTEUR`, `VITESSE` et `FLIR` pour le drone Arma 3, `ALLER`, `ROULER` et `VITESSE` pour le rover Arma 3, `SUIVANT` quand XERULITH a préparé un plan de vol ou un parcours, `REGARDER` selon la tête (complet, ou haut / bas / centre pour une caméra seulement inclinable), `ACTION` s'il existe des gestes ;
- la mission, le format de réponse imposé et les règles de prudence.

Rédiger une bonne **description** de profil est donc la première façon de « programmer » le modèle : elle lui dit quel corps il pilote. Exemple : « rover martien à six roues (suspension rocker-bogie), caméra inclinable à l'avant, télémètre à ultrasons et deux détecteurs d'obstacle infrarouges ».

5.5 Format de réponse et directives

```
PENSEE: Le passage à gauche semble libre, je tourne puis j'avance prudemment.
TOURNER: gauche 1
AVANCER: 2
NOTE: carton bleu à droite du départ
```

Directive	Bornes et effet
`AVANCER: n` / `RECULER: n`	1 ≤ n ≤ pas par directive (3 par défaut) ; refusé si obstacle devant (AVANCER)
`TOURNER: gauche|droite n`	rotations sur place ; odométrie : n × rotation d'un pas
`MONTER: n` / `DESCENDRE: n`	sous-marins et drones
`ALLER: est nord [v]`	drone Arma 3 : point en mètres depuis le départ, dans `rayon_max_m` ; `ALLER: depart` ; v en m/s, « km/h » converti, « max » = vitesse maximale
`VOLER: cap distance [v]`	drone Arma 3 : distance en mètres au cap vrai
`ROULER: cap distance [v]`	rover Arma 3 : même analyse que VOLER (distance en mètres au cap vrai, par la route de l'IA conductrice)
`SUIVANT`	drone et rover Arma 3 : prochain point du plan de vol ou du parcours préparé par XERULITH (missions incendie et rover) ; refusé sans plan
`HAUTEUR: m`	drone Arma 3 : 5 m à `hauteur_max_m` ; refusé pour le rover
`VITESSE: m/s | max`	drone Arma 3 : vitesse des trajets suivants (pas de pas décompté)
`FLIR: on|off`	drone Arma 3 : caméra thermique ; l'observation signale alors les points chauds et leur position au sol
`REGARDER: gauche|droite|centre|haut|bas`	tête à ±balayage, +25° / −15° ; refusé sans tête
`TETE: Δpan Δtilt`	dialogue : déplacement relatif borné à ±60° / ±40°
`DIRE: texte`	synthèse vocale (haut-parleur du robot ou du PC)
`NOTE: texte`	mémoire de mission, avec la position estimée
`ACTION: nom [xN]`	geste de la bibliothèque (N ≤ 5) ; en exploration, gestes seulement
`POSE`, `YEUX`, `PINCE`	dialogue : pose, animation des yeux, pince gauche / droite 0-100
`ATTENDRE: s`	pause de 0 à 30 s
`FIN: bilan`	termine la mission

Directive	Bornes et effet
`STOP`	arrêt immédiat

L'analyse est tolérante : préfixes de liste (`-`, `1.`), gras Markdown, majuscules / minuscules, `TÊTE` ou `TETE`. Une directive inconnue, hors bornes ou interdite est conservée avec la raison du refus (signalée par ⚠️ dans le plan), puis renvoyée au modèle dans l'observation suivante.

5.6 Garde-fous de l'exploration

- 4 directives au plus par cycle ; budget total de pas (`max_steps`) décompté sur avancer, reculer, tourner, monter et descendre, et d'un pas par ALLER, VOLER ou HAUTEUR.
- Drone Arma 3 : point hors du rayon maximal autour du départ, hauteur hors limites, ou vol arcade → ALLER, VOLER, HAUTEUR refusés.
- Rover Arma 3 : MONTER, DESCENDRE et HAUTEUR refusés ; point en mer refusé ; trajet bloqué → contournement automatique, puis alerte au pilote ; ALERTE PENTE dans l'observation.
- Obstacle devant : objet détecté au centre de l'image (plus de 25 % de la surface, bas de l'image) ; télémètre sous `distance_arret_cm` ; détecteur d'obstacle déclenché → `AVANCER` refusé.
- Robot bloqué : image inchangée après une avancée (moins de 1,5 % de pixels modifiés), ou blocage signalé par le pilote → alerte au cycle suivant.
- Batterie sous le seuil du panneau → fin de mission annoncée à voix haute.
- Aucune acrobatie en exploration ; mode **Confirmer** : chaque plan est validé par l'opérateur.

5.7 Modèle de vision

À chaque cycle, le panorama (gauche | centre | droite, ou une seule vue) est envoyé au modèle de vision avec une consigne fixe : décrire en trois phrases les objets et personnes, leur position et leur distance approximative, et dire si la voie devant est libre. La description ouvre la perception transmise au pilote. L'option **Joindre le panorama au modèle de décision** envoie aussi l'image au pilote (il doit alors être multimodal).

6. Les missions d'exploration

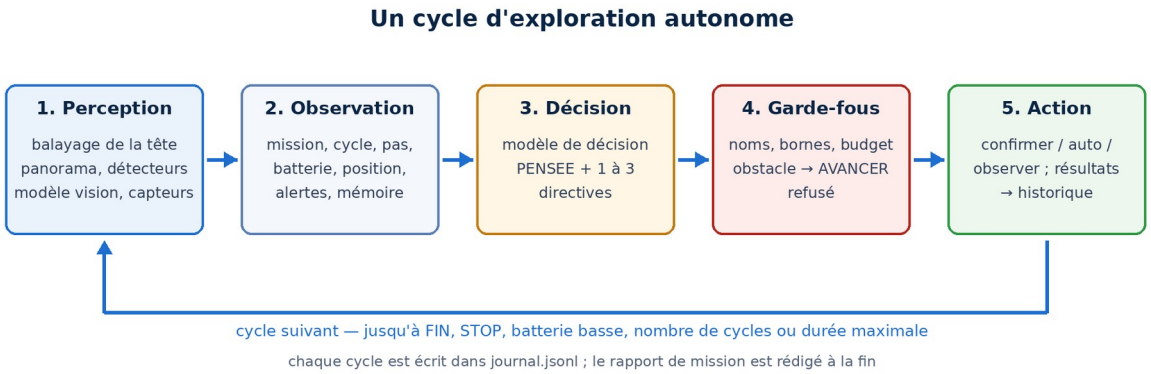


Figure 4 — Déroulement d'un cycle d'exploration.

6.1 Bibliothèque de missions

Les missions sont rangées dans `xerulith_data\missions.json` (créé au premier lancement) ; le panneau d'exploration les ajoute, les remplace (même titre) ou les supprime, et le bouton **Fichier** ouvre le fichier dans l'éditeur de texte (⟳ pour recharger).

```
{
  "_comment": "Missions d'exploration XERULITH : categorie, titre, texte.",
  "missions": [
    {
      "categorie": "Découverte",
      "titre": "Explorer et décrire",
      "texte": "Explore la pièce prudemment et décris ce que tu trouves. Note les
objets et repères remarquables."
    }
  ]
}
```

6.2 Missions fournies

Catégorie	Titre	Texte de la mission
Découverte	Explorer et décrire	Explore la pièce prudemment et décris ce que tu trouves. Note les objets et repères remarquables.
Découverte	Inventaire de la pièce	Fais l'inventaire de la pièce : tourne sur toi-même par petits pas sans avancer, note chaque objet avec sa direction, puis termine avec la liste classée de gauche à droite.
Découverte	Objet le plus grand	Cherche l'objet le plus grand de la pièce, approche-toi prudemment à environ 30 cm et décris-le en détail, puis termine.
Découverte	Tour de la zone	Fais le tour de la zone libre autour de toi et reviens près du point de départ.
Recherche et sauvetage	Recherche de victime (spirale)	Tu es un robot de recherche de victimes. Explore la zone en spirale : avance de 2 pas, tourne à droite, recommence en allongeant progressivement. Dès que tu vois une personne ou un visage, arrête-toi, note sa position, dis « Victime localisée, ne bougez pas, les secours arrivent », puis termine.
Recherche et sauvetage	Balise de détresse ArUco 7	Une balise de détresse est représentée par le marqueur ArUco numéro 7. Trouve-la. Les autres marqueurs sont des fausses pistes : note-les sans t'arrêter. Quand tu vois le 7,

Catégorie	Titre	Texte de la mission
		approche-toi, dis « Balise trouvée » et termine en donnant ta position estimée.
Recherche et sauvetage	Reconnaissance de zone sinistrée	Fais une reconnaissance de zone sinistrée : observe dans toutes les directions, note chaque obstacle et chaque objet avec sa position, sans jamais t'approcher à moins de 30 cm. Termine par un bilan des zones libres et encombrées.
Navigation par repères	Parcours ArUco 1 → 4	Des marqueurs ArUco numérotés de 1 à 4 sont posés dans la pièce. Rejoins-les dans l'ordre 1, 2, 3, 4 et dis le numéro à voix haute quand tu en atteins un. Si tu perds le suivant de vue, tourne sur place pour le retrouver. Termine au marqueur 4.
Navigation par repères	Aller-retour au marqueur 1	Va jusqu'au marqueur ArUco 1, puis reviens au point de départ en te servant de ta position estimée. Dis ton écart estimé à l'arrivée et termine.
Navigation par repères	Recenser les marqueurs	Cherche les marqueurs ArUco et note leur numéro et leur position. Termine quand tu en as trouvé 3.
Interaction humaine	Robot d'accueil	Tu es un robot d'accueil. Reste sur place et tourne lentement la tête. Quand tu vois quelqu'un, salue-le (ACTION: Wave), présente-toi en une phrase comme XERULITH, robot explorateur, puis attends 5 secondes. Termine après avoir accueilli 3 personnes.
Interaction humaine	Cache-cache	Joue à cache-cache : une personne se cache, tu la cherches en commentant à voix haute (chaud, froid…). Quand tu la vois, dis « Trouvé ! », fais une révérence (ACTION: Bow) et termine.
Interaction humaine	Chercher une personne et saluer	Cherche une personne. Quand tu vois un visage, salue-la (ACTION: Wave) et dis bonjour, puis termine.
Surveillance	Gardien de nuit	Tu es un gardien de nuit. Reste immobile (n'avance jamais) et regarde à gauche, au centre puis à droite à chaque cycle. Si quelque chose a changé par rapport au cycle précédent, dis « Mouvement détecté », décris le changement, puis reprends la surveillance.
Surveillance	Objet rouge	Cherche un objet rouge, approche-toi à environ 20 cm, puis termine.
Tests de comportement	Contournement d'obstacle	Avance tout droit jusqu'à rencontrer un obstacle, puis contourne-le par le côté le plus dégagé et reprends ta direction initiale. Termine après l'avoir dépassé.
Tests de comportement	Labyrinthe (main droite)	Tu es dans un labyrinthe en cartons. Applique la règle de la main droite : garde toujours un mur à ta droite. Note chaque intersection. Termine quand tu vois le marqueur ArUco de sortie (numéro 9).
Tests de comportement	Règles conditionnelles	Danse (ACTION: Disco Dance) quand tu vois un visage, recule quand tu vois un obstacle proche, et explore sinon. Termine après 10 cycles.
Drone Arma 3 (formation)	Reconnaissance 1 km² et retour	Mission de reconnaissance par drone d'une zone de 1 km². La zone est un carré de 1000 m de côté qui part du point de départ vers le nord : de E -500 à E +500 et de N 0 à N +1000 (mètres). Plan de vol en passes nord-sud espacées de 200 m, à suivre DANS L'ORDRE, un point par cycle : P1 ALLER: -500 0 | P2 ALLER: -500 500 | P3 ALLER: -500 1000 | P4 ALLER: -300 1000 | P5 ALLER: -300 500 | P6 ALLER: -300 0 | P7 ALLER: -100 0 | P8 ALLER: -100 500 | P9 ALLER: -100 1000 | P10 ALLER: 100 1000 | P11 ALLER: 100 500 | P12

Catégorie	Titre	Texte de la mission
		ALLER: 100 0 | P13 ALLER: 300 0 | P14 ALLER: 300 500 | P15 ALLER: 300 1000 | P16 ALLER: 500 1000 | P17 ALLER: 500 500 | P18 ALLER: 500 0 | P19 ALLER: depart. Au premier cycle : HAUTEUR: 60, puis ALLER: -500 0. À chaque point : regarde l'observation. Pour chaque élément remarquable (bâtiment, véhicule, personne, route, pont, cours d'eau, feu, fumée, zone détruite ou inondée, obstacle au vol), écris une NOTE qui dit ce que c'est et de quel côté. Puis écris « NOTE: P<n> fait » et ALLER vers le point suivant. Ne saute aucun point : ta mémoire (NOTE) dit quel est le dernier point fait. Si le télémètre signale un obstacle à moins de 50 m devant, HAUTEUR: 90 avant de repartir. Après P19 (retour au départ) : HAUTEUR: 10, puis FIN: avec la liste des éléments repérés et leur position (E, N).
Drone Arma 3 (formation)	SATER : recherche de balise 121,5 MHz	Exercice SATER. Un aéronef léger s'est posé en catastrophe : sa balise de détresse (ELT) émet sur 121,5 MHz quelque part dans un rayon de 1500 m autour du point de départ. Tu pilotes le drone de recherche, équipé d'un radiogoniomètre 121,5 MHz. Chaque observation donne le RELÈVEMENT de la balise (cap vrai vers elle ; ±10° quand elle est en vue directe, beaucoup moins précis si le relief la masque) et le NIVEAU reçu en dBm (plus il est fort, plus tu es près : environ -60 dBm à 1500 m, -50 à 500 m, -40 à 150 m, -30 à 50 m). Méthode (ralliement par relèvements successifs) : 1. Premier cycle : HAUTEUR: 80 pour capter au-dessus du relief. 2. Niveau plus faible que -50 dBm : VOLER: <relèvement> 300. 3. Niveau entre -50 et -40 dBm : VOLER: <relèvement> 100. 4. Niveau plus fort que -40 dBm : HAUTEUR: 30, puis VOLER: <relèvement> 40 et REGARDER: bas pour chercher l'épave. 5. Si le relèvement tourne brusquement de plus de 90° ou devient instable, tu viens de passer au-dessus de la balise : VOLER: <relèvement> 20 et REGARDER: bas. 6. Pas de signal, ou un niveau qui baisse deux cycles de suite : HAUTEUR: 120 et reprends au relèvement. À chaque cycle, écris « NOTE: relèvement <x>° niveau <y> dBm » pour pouvoir recouper. Quand tu vois l'épave (hélicoptère détruit) ou le blessé, ou que le niveau dépasse -30 dBm : NOTE: épave localisée (avec ta position E, N), DIRE: Balise localisée, puis ALLER: depart, ensuite HAUTEUR: 10 et FIN: avec la position de l'épave, ce que tu as vu et le nombre de cycles de recherche.
Drone Arma 3 (formation)	Surveillance incendie FLIR 20 km²	Mission de surveillance incendie par drone en caméra thermique FLIR. Un départ de feu a pu se déclarer dans une zone de 20 km² : un rectangle de 5 km sur 4 km au nord du point de départ, de E -2500 à E +2500 et de N 0 à N +4000 (mètres). Tu dois quadriller la zone, détecter le départ de feu et le signaler en donnant sa position. XERULITH a préparé le plan de vol : quadrillage en passes nord-sud espacées de 1000 m, un point tous les 500 m, points en mer retirés (pas de risque de feu en mer, la mer n'est jamais survolée). Tu n'as pas à calculer de points : la directive SUIVANT rejoint le prochain point du plan. Premier cycle, exactement : FLIR: on, HAUTEUR: 150, VITESSE: 30 km/h, SUIVANT. Ensuite, à chaque cycle : lis l'observation (en caméra thermique le blanc est chaud). S'il n'y a pas d'« ALERTE POINT CHAUD », réponds simplement SUIVANT. Réponds court : une phrase de PENSEE et tes directives, sans refaire le plan. Si l'observation contient « ALERTE POINT CHAUD » avec une position au sol : NOTE: point chaud suspect, puis la directive ALLER donnée dans l'alerte, HAUTEUR: 80 et REGARDER:

Catégorie	Titre	Texte de la mission
		bas pour confirmer. Le feu est confirmé si le point chaud reste visible au même endroit ou si la description montre des flammes ou de la fumée. Alors : NOTE: FEU CONFIRMÉ (la position mesurée du point chaud est ajoutée à la note), DIRE: Départ de feu détecté, position transmise, puis ALLER: depart. Si ce n'est pas un feu (véhicule, bâtiment chaud, reflet), note-le, remets HAUTEUR: 150 et REGARDER: centre, puis SUIVANT. Carburant sous 20 % : ALLER: depart immédiatement. Quand l'observation indique « plan de vol terminé » : ALLER: depart, HAUTEUR: 10, puis FIN: avec la position du feu (E, N, latitude, longitude), le cycle de détection et ce que tu as observé, ou « aucun départ de feu détecté ».
Rover UGV Arma 3 (formation)	Exploration terrestre rover UGV 1 km²	Mission d'exploration terrestre par rover UGV (véhicule sans équipage), comme un rover d'exploration planétaire. Zone à explorer : un carré de 1 km² au nord du point de départ, de E -500 à E +500 et de N 0 à N +1000 (mètres). 5 objets à identifier (matériel abandonné, épaves, installations) ont été déposés au hasard dans la zone : trouve-les, approche-les à moins de 15 m pour les identifier et rends compte de leur nature et de leur position. XERULITH a préparé le parcours : passes nord-sud espacées de 150 m, un point tous les 250 m, points en mer retirés (le rover ne roule pas dans l'eau). La directive SUIVANT rejoint le prochain point du parcours ; l'IA conductrice choisit le chemin et contourne les obstacles. Tu n'as pas à calculer de points. Premier cycle, exactement : VITESSE: 15 km/h, SUIVANT. Ensuite, à chaque cycle : lis l'observation. Si le détecteur d'objets ne signale aucun objet « À IDENTIFIER », réponds simplement SUIVANT. Réponds court : une phrase de PENSEE et tes directives. Si le détecteur signale un objet « À IDENTIFIER » : ALLER vers sa position (la directive est donnée dans l'observation). Quand l'observation indique que l'objet est à moins de 15 m : NOTE: SITE <numéro> identifié : <nature de l'objet> (la position mesurée est ajoutée à la note), puis SUIVANT. Si un trajet est « bloqué » (obstacle, pente, eau) : NOTE: passage bloqué, puis SUIVANT (le point est abandonné), ou ROULER: <cap> 50 pour te dégager avant. Si l'observation contient « ALERTE PENTE » : ne poursuis pas dans cette direction, ROULER: <cap opposé> 30, puis SUIVANT. Carburant sous 20 % : ALLER: depart immédiatement. Quand les 5 objets sont identifiés, ou quand l'observation indique « parcours TERMINÉ » : ALLER: depart, puis FIN: avec la liste des objets identifiés (numéro, nature, position E, N, latitude, longitude), les passages bloqués et ce que tu as observé du terrain.

6.3 Écrire une mission

Le texte de la mission est inséré tel quel dans l'invite système, à chaque cycle. Un bon texte de mission tient en trois à six phrases et contient :

1. **un objectif** observable : ce qu'il faut trouver, atteindre ou surveiller ;
2. **une méthode** si elle compte : spirale, main droite, tour sur place, immobilité ;
3. **des contraintes** : distance minimale, zones interdites, nombre de pas, n'avancer jamais ;
4. **des réactions** : « dès que tu vois…, dis… », « si…, alors… » ;
5. **une condition de fin** explicite : « puis termine », « termine après 3 personnes », « termine au marqueur 4 ».

Procédé	Formulation qui marche
Mémoire	« Note chaque objet avec sa direction » : le modèle utilise NOTE, gardée 12 entrées
Comptage	« Termine après avoir accueilli 3 personnes » : il s'appuie sur ses NOTE
Repères	Marqueurs ArUco (dictionnaire 4×4_50) numérotés : détectés localement, sans dépendre du modèle de vision
Parole	« dis « Victime localisée » » : la phrase exacte est reprise dans DIRE
Gestes	« fais une révérence (ACTION: Bow) » : nommer l'action exacte
Prudence	« sans jamais t'approcher à moins de 30 cm » ; « n'avance jamais »
Navigation GPS	« rejoins le point situé à 30 m au nord » : la position GPS et le cap sont dans l'observation
Drone Arma 3	points en mètres est / nord depuis le départ (`ALLER: -500 1000`), un trajet par cycle, « NOTE: P<n> fait » pour suivre le plan, `VITESSE:` au premier cycle
Balise SATER	un texte qui cite « balise » et « 121,5 » fait placer la balise au démarrage ; « rayon de 2 km » fixe le rayon (borné au rayon du profil moins 100 m)
Incendie FLIR	un texte qui cite « départ de feu » (ou « incendie ») et « FLIR » (ou « thermique ») fait placer un feu au hasard dans la zone écrite « de E -2500 à E +2500 et de N 0 à N +4000 » ; avec un plan de vol (`SUIVANT`), le feu est placé près d'un point de passage, toujours détectable
Plan de vol / parcours	avec une zone écrite, « espacées de 150 m » et « tous les 250 m » règlent le quadrillage préparé par XERULITH ; le pilote répond `SUIVANT`
Rover : objets à identifier	pour le rover, un texte qui cite « objets à identifier » (ou « exploration ») fait placer au hasard « 5 objets » (1 à 10) dans la zone écrite ; le pilote les identifie par « NOTE: SITE <n> identifié : <nature> »

6.4 Observation reçue par le pilote

```
OBSERVATION — cycle 4/30, 2.1 min écoulées, pas utilisés 9/60, batterie 7.62 V
Position estimée : x=-12 cm, y=+31 cm (33 cm du départ), cap -30°
Perception :
Description visuelle : Une chaise rouge à gauche à environ 1 m ; sol libre devant.
- vue gauche : objets : chair (81%, 22% de l'image)
- vue centre : rien de détecté par les détecteurs
- vue droite : marqueurs ArUco #7
Capteurs :
- Ultrason avant : 84.00 cm
- Infrarouge gauche : libre
Alertes :
- AVANCER: 3 exécuté MAIS l'image n'a pas changé : bloqué ?
Mémoire (tes NOTE) :
- chaise rouge à gauche du départ (vers x=+0, y=+0 cm)
Derniers cycles :
Cycle 3 : Le passage est libre, j'avance. | AVANCER: 3 : fait
Que fais-tu ?
```

Pour le drone Arma 3, la position est **mesurée** et le radiogoniomètre s'ajoute pendant un exercice SATER :

```
OBSERVATION — cycle 7/60, 6.4 min écoulées, pas utilisés 6/80, batterie ?
Position mesurée : E +412 m, N +655 m, à 774 m du départ (cap pour revenir 212°), cap
actuel 033°, hauteur 80 m
Radiogoniomètre 121,5 MHz : relèvement 058° (25° sur ta droite), niveau -49 dBm
Perception :
Description visuelle : Collines sèches, une piste vers la droite, aucune épave
visible.
Capteurs :
- Télémètre avant (150 m) : —
Mémoire (tes NOTE) :
- relèvement 052° niveau -53 dBm (au point E +260 m, N +420 m)
```

Pour le rover Arma 3, l'observation donne l'assiette, le parcours et le détecteur d'objets :

```
Position mesurée : E -262 m, N +505 m, à 567 m du départ (cap pour revenir 117°), cap
actuel 090°, assiette : tangage +3°, roulis -2°
Parcours (géré par XERULITH) : 9/35 points faits ; prochain point 10 : E -286 m, N
+750 m, à 246 m. Sans alerte, réponds SUIVANT.
Détecteur d'objets embarqué :
- objet n°1 (caisse de matériel) vu à 38 m, relèvement 137°, position estimée E -262
m, N +544 m : À IDENTIFIER — pour l'approcher : ALLER: -262 544
Objets identifiés : n°5 (1 sur 5).
```

6.5 Exemples par type de robot

Rover (GalaxyRVR, rover à roues)

> Explore la pièce en restant à plus de 20 cm des obstacles. Avance par petits pas, et si un détecteur infrarouge signale un obstacle, recule d'un pas puis tourne du côté libre. Note chaque objet remarquable avec sa direction. Termine après 15 cycles par la liste des objets vus.

Tourelle de caméra fixe (surveillance)

> Tu es une caméra de surveillance fixe : n'avance jamais. Regarde à gauche, au centre puis à droite. Si une personne apparaît, dis « Présence détectée », note sa position, puis reprends la surveillance. Termine après 20 cycles par un bilan des passages.

Sous-marin (ArduSub)

> Inspecte le fond du bassin : descends jusqu'à 1,5 m de profondeur par petits paliers, puis avance lentement en ligne droite. Note tout objet posé au fond avec la profondeur. Remonte à 0,5 m et termine après 12 cycles.

Bateau (ArduPilot)

> Fais une ronde le long de la berge : garde le cap au nord pendant 5 pas, tourne de 90° à droite, et recommence pour décrire un carré. Signale toute embarcation ou personne dans l'eau avec ta position GPS. Termine en revenant près du point de départ.

Drone Arma 3

> Survole la route au nord : HAUTEUR: 40 et VITESSE: 8 au premier cycle, puis ALLER: 0 300, ALLER: 0 600, ALLER: 0 900, un point par cycle. À chaque point, note les véhicules et les personnes avec leur position. Reviens au départ (ALLER: depart), HAUTEUR: 10, et termine par la liste des observations.

Les missions fournies de la catégorie **Drone Arma 3 (formation)** (§ 6.2) montrent un plan de vol complet (reconnaissance de 1 km² en 18 points), une méthode de ralliement radiogoniométrique (SATER) et une surveillance incendie au plan de vol préparé par XERULITH (`SUIVANT`). Le rapport d'une mission SATER ajoute une **évaluation instructeur** : distance finale et plus courte distance à la balise, réussite sous 50 m.

Rover UGV Arma 3

> Rejoins le pont au nord-est du départ : VITESSE: 12 km/h au premier cycle, puis ALLER: 300 400. Si le trajet est bloqué, essaie ROULER: 045 100. Au pont, regarde à gauche et à droite et note l'état de la chaussée et la présence d'eau. Reviens au départ (ALLER: depart) et termine par un bilan du trajet et des pentes rencontrées.

La mission fournie **Exploration terrestre rover UGV 1 km²** (catégorie *Rover UGV Arma 3 (formation)*) fait déposer 5 objets au hasard ; XERULITH prépare le parcours (7 passes, 35 points) et le pilote enchaîne `SUIVANT`, `ALLER` vers chaque objet signalé et `NOTE: SITE <n> identifié`. Le rapport ajoute une **évaluation instructeur** : tableau des objets (vu, plus courte distance, signalé, écart), parcours fait, trajets bloqués, distance parcourue et plus forte inclinaison.

JD-Humanoid

> Cherche une personne en tournant la tête. Quand tu vois un visage, salue-la (ACTION: Wave), présente-toi en une phrase, puis termine. N'avance pas de plus de 2 pas d'affilée.

6.6 Réglages de conduite

Réglage	Défaut	Rôle
Mode	Confirmer chaque plan	Confirmer, Autonome, ou Observer (le robot ne bouge pas)
Cycles max	30	nombre maximal de cycles
Pas max (total)	60	budget de pas de la mission
Pas max par directive	3	borne de AVANCER, RECULER, TOURNER…
Durée max	20 min	durée maximale
Batterie mini	7.5 V (JD)	fin de mission sous ce seuil ; profil : batterie minimale
Balayage de la tête	oui	panorama gauche / centre / droite à chaque cycle
Prononcer les pensées	non	le robot dit sa pensée à chaque cycle
Un pas, une rotation	3 cm, 15° (JD)	odométrie ; profil : `pas_cm`, `rotation_deg`

6.7 Journal et rapport

Chaque mission crée `xerulith_data\explorations\AAAAMMJJ_HHMMSS\` :

Fichier	Contenu
`config.json`	les réglages de la mission
`journal.jsonl`	une ligne JSON par événement : `type` (debut, decision, resultat, erreur, fin), `t` (s), `cycle`, puis `observation`, `reponse`, `directives`, `resultats`, `odometrie`, `raison`, `notes`
`cycle_NNN.jpg`	le panorama de chaque cycle
`rapport.md`	rapport Markdown rédigé par le modèle de décision : Mission, Déroulement, Observations remarquables, Bilan (version de secours sans IA)

Le journal se prête à l'analyse hors ligne (Python, tableur) : comparer des modèles sur une même mission, mesurer le taux de directives refusées, rejouer une décision.

7. Programmation par scripts Python

L'onglet **Script Python** exécute un programme Python complet, sans restriction, dans un fil à part. L'objet `robot` traduit chaque appel en coroutine pour le robot actif, quel que soit son profil. **STOP** interrompt le script au prochain appel de `robot` ; `print` écrit dans la console du panneau ; les scripts se rangent dans `xerulith_data\scripts`.

7.1 Référence de l'API

Appel	Effet	Robots
`calibrer()`	calibrage (servos au centre)	JD, EZ-B avec calibrage
`avancer(n)` `reculer(n)` `gauche(n)` `droite(n)`	déplacement de n pas selon le profil	tous
`monter(n)` `descendre(n)`	vertical	sous-marin
`action(nom, boucles=1, vitesse=1.0)`	action de la bibliothèque	JD, EZ-B actions
`pose(nom)`	pose de la bibliothèque	JD, EZ-B actions
`servo(port, degres, vitesse=3)`	un servo, course du profil	EZ-B
`tete(pan=0, tilt=0)`	tête, déplacement relatif	tête ou tourelle
`regarder(pan=0, tilt=0)`	tête, degrés par rapport au centre	tête ou tourelle
`pince(cote, ouverture)`	« gauche », « droite », « les deux », 0-100	JD
`yeux(animation)` `yeux_couleur(r, g, b)`	yeux RGB	JD
`dire(texte)`	synthèse vocale (attend la fin)	tous
`attendre(s)`	pause interruptible par STOP	tous
`batterie()`	tension en volts	tous
`capteurs()`	dictionnaire nom → mesure de chaque capteur du profil	tous
`distance()`	plus petite distance mesurée (cm) ou None	avec télémètre
`adc(port)` `numerique(port)`	entrées de l'EZ-B	EZ-B
`image()`	dernière image JPEG de la caméra (bytes)	avec caméra
`voir(mode)`	analyse : couleur, qr, visages, aruco, mouvement, objets	avec caméra
`armer()` `desarmer()`	véhicule ArduPilot	MAVLink
`profil()`	copie du profil actif (dictionnaire)	tous
`position(port)` `arret()`	dernière position connue d'un servo ; STOP	EZ-B ; tous

`voir()` renvoie `{"mode", "items", "texte", "ms"}` ; chaque élément de `items` porte selon le mode `label` et `score` (objets), `box` (x0, y0, x1, y1 normalisés 0-1), `center`, `id` (ArUco) ou `value` (QR). En cas de problème, la clé `erreur` explique la cause.

7.2 Exemples

Patrouille d'un rover avec évitement

```
robot.dire("Patrouille en cours.")
for tour in range(20):
    d = robot.distance()
    m = robot.capteurs()
    ir = m.get("Infrarouge gauche") or m.get("Infrarouge droit")
    if ir or (d is not None and d < 25):
        robot.reculer(1)
        robot.droite(2)
        continue
    robot.avancer(1)
print("Batterie : %.2f V" % robot.batterie())
```

Tourelle : chercher un visage

```
for pan in (-40, -20, 0, 20, 40):
    robot.regarder(pan=pan)
    robot.attendre(0.6)
    r = robot.voir("visages")
    if r["items"]:
        robot.dire("Je vous vois.")
        break
robot.regarder(0, 0)
```

Sous-marin : palier de profondeur

```
robot.armer()
while (robot.capteurs().get("Profondeur") or 0) < 1.5:
    robot.descendre(1)
    robot.attendre(1)
robot.avancer(3)
robot.monter(2)
robot.desarmer()
```

Rover : suivre les marqueurs ArUco 1 à 4

```
for cible in (1, 2, 3, 4):
    for essai in range(12):
        ids = [i["id"] for i in robot.voir("aruco")["items"]]
        if cible in ids:
            robot.dire("Marqueur %d." % cible)
            robot.avancer(2)
            break
        robot.gauche(1)
```

8. Commandes vocales

Les phrases reconnues par le micro du PC (faster-whisper, modèle « small », français) sont comparées à `xerulith_data\commandes_vocales.json`, relu à chaque phrase. Une phrase sans correspondance peut être envoyée à l'IA (case du panneau Voix).

```
{
  "commandes": [
    {
      "phrases": ["avance", "marche", "en avant"],
      "faire": "ACTION:Forward",
      "repondre": "J'avance."
    },
    {
      "phrases": ["stop", "arrête"],
      "faire": "STOP",
      "repondre": "Je m'arrête."
    },
    { "phrases": ["batterie"], "faire": "BATTERIE", "repondre": "" }
  ]
}
```

Valeur de faire	Effet
`STOP`	STOP général
`CALIBRER`	calibrage
`POSE:nom`	pose de la bibliothèque
`ACTION:nom`	action de la bibliothèque
`BATTERIE`	annonce la tension

9. Sécurité

Règle	Mise en œuvre
STOP partout	bouton, touche Échap (raccourci global), commande vocale, directive ; arrête mouvement, script, exploration, directives de l'IA et son
Servos tenus	STOP et déconnexion ne relâchent jamais les servos ; Relâcher demande une confirmation
Courses	positions hors course refusées, jamais rognées (JD : courses de Commander ; autres : courses du profil)
Calibrage	obligatoire avant tout mouvement pour le JD et les profils qui l'exigent
Batterie	JD : 7,20 V (tout mouvement) et 7,45 V (marche, acrobaties) ; autres : `batterie_min` du profil
Mouvements bornés	chaque pas a une durée ; les moteurs reviennent au neutre à la fin et sur STOP
ArduPilot	armement explicite avec confirmation ; consignes RC rendues à la radiocommande après chaque pas ; sécurités ArduPilot prioritaires
GalaxyRVR	état renvoyé toutes les 0,3 s ; coupure Wi-Fi de 3 s → arrêt du rover par son ESP32
Carte maison	la carte coupe ses moteurs à la fin de la durée reçue (à programmer dans le croquis)
Drone Arma 3	pont local, ordres bornés, client perdu = arrêt du drone, zone et hauteur maximales du profil ; aucune conséquence matérielle : banc d'essai des nouvelles missions en mode Autonome
Exploration	au sol pour un robot terrestre, zone dégagée, opérateur à portée ; mission d'abord mise au point en simulation, puis mode Confirmer sur la plateforme réelle

10. Fichiers et réglages

Élément de xerulith_data	Contenu
`xerulith.ini`	réglages : fenêtre, profil actif, adresses par profil, IA, voix, exploration
`profiles\*.json`	un profil de robot par fichier ; `.profils_fournis_v1` : profils fournis déjà livrés
`missions.json`	bibliothèque de missions
`commandes_vocales.json`	commandes vocales
`explorations\`	une mission par dossier
`scripts\`, `captures\`	scripts Python ; images de la caméra
`logs\`	journal du jour, `console.log`, `crash.log`

Fichier du pont Arma 3 (arma3, à côté de l'exe)	Destination et rôle
`@XerulithBridge\`	dossier d'Arma 3 (à côté de `arma3_x64.exe`) ; mod à charger avec `@Pythia`
`mission_drone\init.sqf`	dossier de la mission (`Documents\Arma 3 - Other Profiles\<profil>\missions\<nom>.<carte>\`) : crée le drone à 10 m et lance le pont
`mission_drone\xerulith_bridge.sqf`	même dossier : le pont côté mission
`LISEZMOI.md`	installation et utilisation du pont

Section [clé] de xerulith.ini	Rôle
`[robot] profile`	identifiant du profil actif
`[robot] ip`, `[profil_adresse] <id>`	adresse du JD ; adresse de chaque autre profil
`[robot] sim`, `eyes_hw`	case Simulateur ; carte des yeux I2C
`[ai] hosts`, `connection`, `model`, `mode`	serveurs, connexion (reseau, cloud, auto), modèle et mode du dialogue
`[ai] key_keep`, `cloud_key`	clé Ollama Cloud chiffrée (DPAPI), seulement si demandée
`[voice] engine`, `voice`, `rate`, `gain`	synthèse vocale
`[explore] …`	mission, modèles, mode, cycles, pas, durée, batterie, étalonnage (valeurs du JD)
`[win] geometry`, `state`	position de la fenêtre et des panneaux

Supprimer `xerulith.ini` remet tous les réglages par défaut au lancement suivant (les profils, missions et explorations sont conservés).

11. Diagnostic

Symptôme	Analyse
L'application se ferme au lancement	`console.log` : la trace `faulthandler` donne la fonction en cause ; essayer `XERULITH.exe` d'une autre installation, ou renommer `xerulith.ini`
« Aucun battement (HEARTBEAT) »	chaîne de connexion, port occupé par Mission Planner ou QGroundControl, débit de la radio
« GalaxyRVR injoignable »	Wi-Fi du rover, application SunFounder encore connectée, interrupteur de téléversement
Capteur « — »	source incompatible avec le contrôleur, clé de carte maison différente, ou pas encore de mesure
Mouvement refusé « hors de sa course »	corriger `min` / `max` du servo dans le profil
Le pilote ne produit aucune directive valide	modèle trop petit ou non conversationnel ; vérifier le format dans `journal.jsonl` (champ `reponse`)
Exploration lente	modèles déchargés à chaque cycle (`ollama ps`), mémoire graphique insuffisante, ou modèle cloud saturé
Objets non détectés	`ssd_mobilenet_v1_12.onnx` absent de `_internal\data\models` ; image trop sombre (luminance moyenne < 20 sur 255)
Reconnaissance vocale grisée	modules vocaux absents de la version installée ; le premier usage télécharge le modèle Whisper (connexion Internet nécessaire une fois)
« Pythia module "XerulithArma.poll" not recognized »	`@XerulithBridge` non chargé ou sans `addons\xerulith_bridge.pbo` (§ 4.6)
Drone Arma 3 figé quand XERULITH a le focus	Arma lancé sans `-noPause`
Capture d'une mauvaise fenêtre	journal « Capture de la fenêtre … » : vérifier le programme retenu ; fenêtre d'Arma recouverte
Erreur dans un script SQF	dernier `.rpt` de `%LOCALAPPDATA%\Arma 3\`

Annexe A. Bibliothèque du JD

Niveau	Actions
Gestes (17)	Bow, Disco Dance, Fly, Grab, Hands Dance, Happy Hands, Head Bob, Pass the Mic, Point, Roll Hands, Singing, Singing Hands In, Singing with Hands, Stop, Thinking, Wave, YMCA Dance
Déplacements (13)	Forward, Gorilla, Head Bob Feet, Jump Jack, Kick, Left, Lunge Singing, Predance, Reverse, Right, Shimmy, Throw Mic, YMCA March
Acrobaties au sol (9)	Getup, Headstand, Pushups, Sit Down, Sit Wave, Situps, Splits, Stand From Sit, Summersault
Animations des yeux (13)	Banana, Big-Small, Diag Scan, Disco, Dots, Expressions, Flash, Scanner, Spin, Spin Roll, Stripes, Test, Wink

Poses principales : `Calibrate` (tous les servos à 90°), `STAND` (debout) ; la bibliothèque compte 126 poses utilisées par les actions.

Annexe B. Crédits et licences

- **XERULITH** est un logiciel libre distribué sous licence **GNU GPL v3** (fichier `LICENSE.txt` du dossier d'installation, accepté à l'installation).
- Protocole EZ-B v4, bibliothèque de mouvements du JD, détecteurs de vision et simulateur : projet **Commander** de Dustin Hightower, licence MIT, utilisé sans modification.
- pymavlink (ArduPilot), LGPL-3.0 ; pyserial, BSD ; websockets, BSD ; OpenCV, Apache 2.0 ; onnxruntime, MIT ; SSD MobileNet v1 (ONNX Model Zoo), Apache 2.0 ; PySide6 (Qt), LGPL-3.0.
- GalaxyRVR : protocole des projets galaxy-rvr et ai-camera-firmware de SunFounder. EZ-Robot, ARC, JD-Humanoid, SunFounder et GalaxyRVR sont des marques de leurs propriétaires ; XERULITH n'est affilié à aucun d'eux.
- Simulation de vol : mod Pythia (github.com/overfl0/Pythia) ; Arma 3 est une marque de Bohemia Interactive, XERULITH n'y est pas affilié.
- Modèles d'IA servis par Ollama : Gemma (Google), Qwen (Alibaba) et modèles du catalogue ollama.com, chacun sous sa propre licence.

XERULITH — by F1GBD (c) 2026